

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_cffc9fdf-8f2\agent-a48690a574d0116e6.jsonl`
- SHA-256: `60547c920ff21c098453f07ea0f2ee7bbc1bdfbeddd4c10a69f9e46659da447b`
- Source modified: `2026-06-12T09:14:06+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_cffc9fdf-8f2`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T09:07:11.874Z)

You are an adversarial verifier. Below are findings from three research agents about the Roora vendor annotation flow across three repos:

The three sibling repos (all are accessible working directories):

- C:\Users\avidu\Projects\badminton-highlight-indexer (main indexer: FastAPI + ffmpeg pipeline)
- C:\Users\avidu\Projects\sports-data-collector (data collection; believed to hold the Roora vendor ingestion flow)
- C:\Users\avidu\Projects\rally-annotator (VLC-based annotation tool)

delay-evidence

Summary: The Roora annotation vendor timing delay is documented in two formal review documents from the badminton-highlight-indexer project's Annotation Setup folder. The issue is specifically documented in the Pickleball & Table Tennis pilot review (2026-06-10), which identifies systematic boundary accuracy problems.

****Documented Offset Measurements:****

- ****Pickleball & Table Tennis videos:**** Every rally boundary (both start and end) was off by ****0.5 seconds or more****, with several rallies showing ****up to ~2 seconds of delay****
- Tolerance requirement: ****±0.3 seconds**** (specified in ROORA_BRIEF.md Brief §4 and ANNOTATION_GUIDE.md Part 3)
- This was identified as a regression from the badminton pilot, which had "generally good" timings

****Direction & Distribution:****

- The issue affects BOTH start and end boundaries (not unidirectional)
- The badminton pilot showed acceptable boundary timing but pickleball/table tennis showed systematic delay across all 12 rallies in those videos

****Root Cause - Documented Hypothesis:****

The systematic cause was identified in the ingestion analysis: All 24 rally boundaries in the pickleball and table tennis videos were ****placed exactly on a 30-frame sampling grid****. At the declared 59.94 fps, this 30-frame grid = ~0.5 seconds apart. This means:

- Boundaries that fall "between grid steps" in the actual footage were incorrectly snapped to the nearest grid frame
- The annotators were sampling every 30th frame INSIDE rallies (acceptable) but the ****first and last boxes of each rally must be placed on exact serve-contact and ball-dead frames**** (Guide Part 3), not snapped to grid
- This sampling-grid alignment explains why the minimum observed offset is ~0.5s (one grid step)

****Impact Analysis (from document):****

- For 1-8.5 second rallies, a 0.5-2s error corrupts ****10-100% of the rally span****
- A 1.0s table tennis rally can be completely erased by a 2s boundary error
- At the standard evaluation gate (IoU ? 50%), a typical 4.5s rally mislabeled by 2s late "fails to match even a perfectly correct detection"
- Produces corrupted golden-set labels that would train models incorrectly and invalidate

quality metrics

****Documented Hypothesis for Cause:****

The root cause document states: "All 24 rally boundaries sit **exactly** on the 30-frame sampling grid ? at 59.94 fps that grid is ~0.5 s apart, so a boundary placed only on grid frames **cannot** meet ± 0.3 s." This indicates the annotators used CVAT's frame sampling at fixed intervals rather than stepping through individual frames to find exact rally boundaries.

****Mitigation/Corrections Documented:****

1. For future batches: "place the first/last box of every rally on the exact contact/dead frame (frame-step to find it ? Guide Parts 3 and 6)"
2. Self-check boundaries against ± 0.3 s before submitting (added to Guide Part 7 checklist)
3. The documents note schema changes to remove confusing fields and clarify frame-stepping requirements

This issue was specific to the pickleball and table tennis pilots (delivered 2026-06-10) and was a regression from the badminton pilot. The badminton review (2026-06-06) explicitly noted "the rally **timings** are generally good" with only minor boundary and completeness issues, not systematic offset problems.

Key files: [{"path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md", "why": "Primary source documenting the systematic 0.5-2s boundary offset in Roora's pickleball and table tennis annotations, the root cause (30-frame sampling grid at 59.94 fps), and impact on golden-set quality", "lines": "1-21, 2-22"}, {"path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Badminton (2026-06-06).md", "why": "Companion review showing badminton had acceptable timing (no systematic delay documented), establishing this was a regression in the later sports", "lines": "1-30"}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\R00RA_BRIEF.md", "why": "Vendor specification defining the ± 0.3 s boundary tolerance requirement (Brief §4) and the format for timing annotations", "lines": "79-87"}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\ANNOTATION_GUIDE.md", "why": "Illustrated guide specifying exact frame-stepping methodology (Part 3, Part 6) and ± 0.3 s boundary tolerance, referenced in the pilot review as the source of requirements", "lines": "83-96, 195-200"}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py", "why": "Evaluation harness that computes boundary errors (mean_start_error, mean_end_error) for any annotation source, used to measure Roora's offset during pilot review", "lines": "104-135"}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md", "why": "Design doc explaining the vendor bake-off process and quality-gate measurement methodology, with Roora selected 2026-06-02 as the pilot vendor", "lines": "1-50"}]

Quotes: ["C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md, §2: 'Every rally's start/end in both videos is off by 0.5 s or more ? up to ~2 s in several ? against the agreed tolerance of ± 0.3 s'", "C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md, §2 Root Cause: 'All 24 rally boundaries sit exactly on the 30-frame sampling grid ? at 59.94 fps that grid is ~0.5 s apart, so a boundary placed only on grid frames cannot meet ± 0.3 s'", "C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md, §2 Impact: 'a 0.5?2 s boundary error corrupts 10?100% of a rally's span; the 1.0 s table-tennis rally #2 can be erased entirely by it'", "C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md, §2: 'at our standard evaluation gate (overlap ? 50%), a typical 4.5 s rally labeled 2 s late fails to match even a perfectly correct detection'", "C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Badminton (2026-06-06).md, §1: 'After a full frame-by-frame review, the rally timings are generally good'", "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\R00RA_BRIEF.md, §4: 'Boundaries: start_time/end_time within ± 0.3 s of true serve-contact / landing'", "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py, lines 124-125: 'mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) ...) / tp' and 'mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) ...) / tp'"]

ffmpeg-tooling

Summary: ## Video Processing & Ffmpeg Tooling Inventory

****Across three repos****, a comprehensive ffmpeg/ffprobe ecosystem exists for video ingestion,

validation, clipping, and compilation. Here's what was found:

KEY FINDING: Video ID / Re-Encoding Risk

- **badminton-highlight-indexer** uses MD5 hash of the **entire file** as `video_id` (`backend/utils/hashing.py` lines 23-33`). A re-encoded copy changes the bytes, hence the MD5, creating a new `video_id` ? this matters for the vendor-prep workflow.
- **Architectural implication:** If Roora (or any vendor) receives a video, re-encodes it to a "vendor-friendly profile," and returns it, that re-encoded copy will have a different MD5. The system currently treats it as a completely different video, which could cause duplicate-handling issues.

1. FFMPEG/FFPROBE INVOCATIONS BY REPO

```
#### badminton-highlight-indexer (Main ffmpeg usage)
- backend/utils/video.py (52 lines):
  - get_video_duration() ? ffprobe only (extracts duration).
  - extract_video_chunk() ? transcodes OR stream-copies clips with optional downscaling.
Parameters: -ss` (seek), -t` (duration), -c:v libx264 -preset ultrafast -crf 23` for re-encode, -c copy` for stream-copy, -vf scale=<width>:-2` for downscaling.

- backend/pipeline/stitcher/compiler.py (224 lines):
  - get_video_duration() ? ffprobe (get duration).
  - compile_highlights() ? two-pass ffmpeg workflow:
    - Trim clips: ffmpeg -ss <start> -to <end> -i <video> -c:v libx264 -preset superfast -crf 22 -c:a aac -b:a 128k` (re-encodes each segment at intermediate quality).
    - Concatenate: ffmpeg -f concat -safe 0 -i clips.txt -c copy` (stream-copies pre-encoded clips).
  - Config: stitching.begin_padding=0.5s`, end_padding=0.5s` (lines 72-77 of config.json).

- backend/pipeline/validators/format.py (87 lines):
  - FormatValidator.validate() ? ffprobe JSON dump to validate MP4 container, extract codec, duration, size.

- backend/pipeline/validators/resolution_fps.py (68 lines):
  - ResolutionFPSValidator.validate() ? validates min resolution (default 1280x720) and min FPS (29.9). Context: ffprobe_meta` from `format.py`.

- backend/pipeline/validators/pts_continuity.py (117 lines):
  - Analyzes keyframe packet timestamps (ffprobe -show_entries packet=pts_time,flags`). Detects splicing/timeline gaps. Thresholds: max gap >10s or backward jumps = fail.

- backend/tools/rally_slicer.py (150 lines):
  - probe_duration() ? ffprobe duration.
  - slice_rallies() ? clips rallies with padding: ffmpeg -ss <start> -to <end> -i <video> -c:v libx264 -preset superfast -crf 22 -c:a aac -b:a 128k`.

- backend/pipeline/audio/hit_detector.py (147 lines):
  - extract_audio() ? audio-only extraction: ffmpeg -i <video> -vn -ac 1 -ar 16000 -c:a pcm_s16le <out.wav>`.

- backend/pipeline/detectors/wasb_infer.py, tracknet_runner.py ? clip staging for inference (likely using utilities above).

#### sports-data-collector (Ingestion/download focused)
- src/core/downloader.py (196 lines):
  - No ffmpeg calls. Uses yt-dlp for YouTube downloads (best-resolution selection: bv*+ba/b` format, -S res,fps,...` sort).
  - probe_resolution_fps() ? ffprobe only: extracts width, height, avg_frame_rate from downloaded video (lines 112-140).
    - Part of the Roora vendor ingestion: videos are downloaded here, then sent to the vendor for annotation. Critical: downloader.py` does NOT re-encode videos before sending to vendors.

- src/cli/curate.py (1093 lines):
  - fetch() command ? calls download_best_video() to re-download/upgrade resolution. Takes
```

```

`--max-height` (cap resolution, default 720p+). No re-encoding or preprocessing.
- Annotation import pipeline: `import-local` registers video + CSV. Schema validation, no
transcoding.
- `convert-cvat` ? normalizes vendor CVAT export (bounding-box XML) to canonical CSV. No video
manipulation.

- **`src/parsers/cvat/rally_cvat.py`** (uses `probe_resolution_fps()` from downloader).

#### **rally-annotator** (VLC-based UI, no ffmpeg)
- No ffmpeg usage found. Pure UI/annotation tool.

```

2. EXISTING PREPROCESSING / PROXY GENERATION

```

**None found.**

- Badminton-highlight-indexer has validators but **no "prep for API inference" downscaling
pipeline.** The `extract_video_chunk(..., scale_width=...)` function exists but is **not used in
the indexing flow** (only for manual clip extraction).
- Sports-data-collector downloads at best resolution (`max_height` cap can lower it, but default
is best available). **No lower-res proxy for UI or inference.**
- No mention of "low-res copies for inference" anywhere.

```

3. CLI TOOLS / SCRIPTS DIRECTORIES

```

| Repo | Location | Contents |
|---|---|---|
| **badminton-highlight-indexer** | `backend/tools/` | `export.py` (segment export to
EDL/CSV/JSON), `rally_slicer.py` (clip individual rallies) |
| **sports-data-collector** | `src/cli/` | `curate.py` (main CLI: status, import-local, fetch,
convert-cvat, validate-delivery, export), `ingest.py`, `normalize_annotations.py` |
| **rally-annotator** | None | ? |

```

```

**Roora vendor workflow currently lives in** `sports-data-collector/docs/annotation/`:
- `ROORA_BRIEF.md` ? vendor spec (CSV schema, rally definition, rules).
- `ANNOTATION_GUIDE.md` ? detailed with examples & pictures.
- Intake manifest template + NDA template (TODOs still unfilled per brief §8).

```

```

**Current flow (per `INGESTION_PIPELINE.md`):**
1. Download video via `fetch` (optional, best resolution).
2. Send to Roora; they return CVAT export or CSV.
3. `convert-cvat` (if needed) ? `validate-delivery` (QA gate) ? human review ? `import-local`
(register in DB) ? `export` (to indexer).
4. **No video preprocessing step.** Roora receives original file as-is.

```

4. VIDEO_ID / HASHING IMPLICATIONS

```

- **Problem statement:** `compute_video_id()` in `backend/utils/hashing.py` does MD5 of entire
file (64 KB chunks). Used as **unique key** in the SQLite DB via
`db_instance.add_video(video_id, path, ...)` (main.py line 168 and 681).
- **Current assumption:** one video file = one video_id. If the indexer re-processes a video, it
re-computes the same MD5 and finds it in the DB (reingest logic, main.py line 139: `was_reingest
= db_instance.get_video(video_id)` is not None).
- **Re-encoding risk:** If you create a lower-res or re-encoded copy for vendor submission, it
gets a different MD5 ? treated as a new video ? potential **duplicate rows in the DB if both the
original and re-encoded copy are indexed.**
- **Example scenario:** Roora could receive a copy downsampled to 720p, re-annotate, and the
annotator returns it along with the CSV. If someone tries to index both the original and the
returned re-encoded copy, they'd collide only by filename/path, not by video_id.

```

5. ENCODING PARAMETERS SUMMARY

Component	Codec	Preset	CRF	Audio	Keyframes	Use Case
***compiler.py (trim)**	libx264	superfast	22	aac 128k	implicit	Intermediate clips before concat
***compiler.py (concat)**	?	?	?	?	?	Stream-copy (no re-encode)
***rally_slicer.py**	libx264	superfast	22	aac 128k	implicit	Standalone rally clips
***extract_video_chunk**	libx264	ultrafast	23	aac	implicit	Low-cost clip extraction
***hit_detector.py**	pcm_s16le (audio)	?	?	16 kHz mono	?	Audio analysis

Keyframe analysis: `pts_continuity.py` detects GOP structure (typical 1?4s intervals for GoPro). No explicit keyframe-interval control in any encoder call (no `-g` flag). Re-encoded clips likely inherit H.264 defaults (~2s @ 30fps = GOP 60).

6. WHERE VENDOR-PREP RE-ENCODE FITS BEST

Recommended: `sports-data-collector` repo, specifically in a `new` `src/core/vendor_prep.py` module or `extend` `downloader.py`.

Rationale:

- Flow:** Vendor videos are sourced and managed here (fetch ? send to Roora ? validate ? import).
- Artifact:** The downloader already probes resolution/fps post-download. A vendor-prep module would:
 - Detect if source video exceeds vendor specs (e.g., >1GB, >1080p, VFR, complex codec).
 - Re-encode to vendor-friendly profile: `1080p max, 30 fps constant, H.264, keyframe every 2s (-g 60 @ 30fps)`.
 - Save alongside original (e.g., `<video_id>.mp4` and `<video_id>_vendor_prep.mp4`).
 - Send vendor-prep copy to Roora; still send original `video_id` for the CSV labels (so MD5 identity is stable).
 - Return CSV points to `original` file (via `video_id`), not the re-encoded copy, to avoid duplicate-video-id issues.

Alternative location: `badminton-highlight-indexer/backend/tools/` if the prep is meant as a `post-indexing` step (e.g., "prepare clips for a third-party API"). Less natural for vendor ingestion flow, but possible.

NOT recommended: `rally-annotator` (VLC UI, no processing logic).

KEY ISSUE FOR ROORA PILOT

The current `ROORA_BRIEF.md` (§1) states: `"Do not trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum."` This is correct for the indexer's MD5-based `video_id`. But if Roora (or any vendor) is supposed to `return` a re-encoded copy, that guidance needs updating. Current design assumption: `vendor` receives original, returns CSV labels only (not a re-encoded video). If that changes, the database schema and `video_id` logic will need a `canonical_video_id` (source file hash) separate from `working_video_id` (current file hash) to track re-encoded variants.

Key files: [{"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\hashing.py", "lines": "1-34", "why": "Canonical MD5 hash of entire file used as video_id (DB key). Critical for understanding re-encoding implications."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\video.py", "lines": "1-52", "why": "Extract chunk with optional re-encode"}]

and downscaling. Defines encoding parameters: libx264, crf 23, scale filter."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\stitcher\\compiler.py", "lines": "1-224", "why": "Highlight compilation: two-pass ffmpeg (trim with libx264 crf 22, concat with stream-copy). Padding: 0.5s begin, 1.0s end."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\validators\\format.py", "lines": "1-87", "why": "Validates MP4 container and extracts ffmpeg metadata (codec, duration, format)."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\validators\\resolution_fps.py", "lines": "1-68", "why": "Validates minimum resolution (1280x720 default) and FPS (29.9 default) from ffmpeg metadata."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\validators\\pts_continuity.py", "lines": "1-117", "why": "Keyframe timestamp analysis via ffmpeg. Detects splicing/timeline gaps. GOP structure analysis."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\tools\\rally_slicer.py", "lines": "1-150", "why": "CLI tool for extracting individual rally clips. Same encoding as compiler (libx264 crf 22, aac 128k)."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\audio\\hit_detector.py", "lines": "1-147", "why": "Audio extraction via ffmpeg (-vn -ac 1 -ar 16000 -c:a pcm_s16le). No video re-encode, audio-only."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\config.json", "lines": "68-78", "why": "Stitcher config: begin_padding 0.5s, end_padding 0.5s. No vendor-prep or transcode settings."}, {"path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md", "lines": "1-191", "why": "Comprehensive vendor selection & workflow doc. Roora chosen for pilot. Mentions video MD5 identity (ADR) and privacy guardrails."}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\core\\downloader.py", "lines": "1-196", "why": "Best-quality video acquisition via yt-dlp (no ffmpeg encoding). Probes resolution/fps post-download. Part of Roora vendor prep."}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\cli\\curate.py", "lines": "963-1093", "why": "CLI entry points for the ingestion pipeline: status, import-local, fetch, convert-cvat, validate-delivery, export. No video re-encode logic."}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\ROORA_BRIEF.md", "lines": "1-132", "why": "Vendor-facing spec. Requires original video (no re-encode): 'Do not trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum.'"}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\INGESTION_PIPELINE.md", "lines": "1-145", "why": "End-to-end ingestion FAQ. Current flow: download ? send to Roora ? validate ? import ? export. No vendor-prep re-encode step."}]

Quotes: ["C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\hashing.py:3-11: 'video_id' is the MD5 of the *whole* video file. Historically each endpoint...defined its own 'get_file_md5' with a different read-chunk size...The hash values agreed (same bytes hashed), but the duplicated code was a footgun: any future change to one copy would silently diverge the DB key.", "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\video.py:20-32: 'extract_video_chunk...scale_width: If set (requires reencode), downscale to this width (height auto, even). Useful for high-res sources headed to an API that analyses at low res anyway ? shrinks uploads with no analysis loss.", "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\stitcher\\compiler.py:139-150: 'cmd = [\"ffmpeg\", \"-y\", \"-ss\", str(round(padded_start, 3)), \"-to\", str(round(padded_end, 3)), \"-i\", raw_video_path, \"-c:v\", \"libx264\", \"-preset\", \"superfast\", \"-crf\", \"22\", \"-c:a\", \"aac\", \"-b:a\", \"128k\", clip_path]\", "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\core\\downloader.py:1-18: 'Best-quality video acquisition. A single place that knows how to fetch the *highest resolution available* for a source URL...We never trust the requested resolution; we ffmpeg the resulting file and record the *actual* width/height/fps as the source of truth.", "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\ROORA_BRIEF.md:52-54: 'Times are relative to the **start of the file we provide**. Do **not** trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum.", "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md:153: '**Roora** (Roora ML Pvt Ltd) | **Bengaluru** ? HSR Layout (verified, 'rooragr.com') | Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot (2026-06-02)** ? existence/capability/location verified; **quality TBD via the pilot**.'"]

handoff-flow

Summary:

End-to-End Roora Vendor Handoff Flow ? Complete Map

1. ANNOTATION GUIDELINE & VENDOR TOOLING

****Guideline Document:****

- Location: `/sports-data-collector/docs/annotation/ROORA\_BRIEF.md` (lines 1-132)
- Also: `/docs/annotation/ANNOTATION\_GUIDE.md` (illustrated supplement with per-sport rules)
- ****Roora Contact:**** `info@rooragrp.com` (Bengaluru, HSR Layout)

****What Roora receives:****

- Annotation brief with rally CSV schema (one row per rally: `rally\_number`, `start\_mmss`, `end\_mmss`, `start\_time`, `end\_time`, `shots\_count`, `ending\_reason`, `last\_shot\_outcome`, `score\_before`, `score\_after`, `notes`)
- Intake manifest with per-video `fps` and `video\_id` (template at `/docs/annotation/INTAKE\_MANIFEST\_TEMPLATE.csv`)
- Video files via Google Drive (to be shared; locations in TODOs §8)
- Blank Google Sheet template with mm:ss?seconds formula pre-filled
- NDA template at `/docs/annotation/NDA\_TEMPLATE.md`

****Vendor's tooling & constraints (from brief & implementation):****

- ****Recommended:**** any tool showing current time (mm:ss.mmm or seconds, or frame + fps) with frame-step capability (VLC, Label Studio, CVAT)
- ****Deliverable format:**** Per-video CSV named exactly `.csv` (matching the supplied video_id)
- ****Format constraint:**** Times are `m:ss.mmm` format for mm:ss columns; decimal seconds auto-fill from formula (or vendor enters decimal seconds directly)
- ****Critical:**** "Do not trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum" (ROORA_BRIEF.md §1, line 52)
- ****Resolution/fps:**** No explicit tool constraint documented; videos are delivered at their native resolution/fps. Vendor receives these via manifest.
- ****Pilot scope:**** 3 videos (badminton, pickleball, table tennis), each <3 min with ~10-12 rallies, as a paid calibration round

2. INGESTION PATH & ANNOTATION->VIDEO JOIN KEY

****Primary join key:**** `(video\_id, rally\_number)`

- Video keyed by human `video\_id` (supplied in intake manifest)
- Rally keyed by 1-based integer `rally\_number` (chronological, no gaps)
- Database schema in `/src/core/db.py` lines 76-88 (badminton_rallies table):
```sql  
PRIMARY KEY (video\_id, rally\_number)  
FOREIGN KEY (video\_id) REFERENCES videos(video\_id)  
```

****Timestamp units & precision:****

- ****Canonical format:**** Decimal ****seconds**** (floats)
- ****Precision:**** ±0.3 s tolerance on boundaries (ROORA_BRIEF.md §4, line 83; ANNOTATION_GUIDE.md Part 3)
- ****Formula source:**** mm:ss.mmm ? seconds conversion via Google Sheet formula (shown in ROORA_BRIEF.md §1, lines 35-36)

****Ingestion pipeline:****

- **CSV parsing**** (`/src/cli/curate.py` lines 216-362):
 - Reads canonical CSV with headers: `rally\_number, start\_time, end\_time, ...` (lowercase-folded)
 - Uses `safe\_float\_required()` to parse start_time/end_time as decimal seconds (required fields; raises if missing)
 - Falls back to mm:ss parsing if decimal is missing: `/src/core/parsing.py` lines 32-49 (`parse\_time\_string`)
- **CVAT alternative**** (if vendor uses CVAT instead of CSV):
 - Converts CVAT XML box export ? canonical CSV via `/src/parsers/cvat/rally\_cvat.py`
 - ****Critical:**** Recomputes times from frame ranges: `seconds = frame / fps` (lines 266-267)
 - ****Ignores vendor's time attributes**** ? a CVAT job that assumes 30 fps yields systematically wrong times (lines 23-24)

```

3. Schema validation (`/src/core/validator.py` lines 29-57):
  - `end_time > start_time` (strictly greater)
  - No overlaps: `end_time[N] <= start_time[N+1]`
  - Chronological by rally_number

4. Annotation objects created as `BadmintonRally` (Pydantic model,
`/src/core/schemas/badminton.py` lines 1-21):
  - Fields: `video_id, rally_number, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after`
  - Validation: `start_time > 0, end_time > start_time`

Critical join behavior:
- Annotations are keyed to the original video file via `video_id`
- The collector DB stores the video's `local_path` (from import command, e.g. `--video-path
match.mp4`)
- Indexer repo retrieves the video's MD5 and local_path from the manifest
(`/data/eval_export/manifest.json`)
- No timestamp offset handling during ingestion ? times are taken as-is from CSV (must be
pre-aligned to the vendor's received video)

---
```

3. TIMESTAMP NORMALIZATION & OFFSET HANDLING

```

Normalization (before import): `/src/cli/normalize_annotations.py`
- Converts mm:ss / HH:MM:SS to decimal seconds
- Drops rows with invalid/missing times or negative start_time
- Detects & renumbers fractional `rally_number` (e.g., 7.5 ? collides on PK) or duplicate IDs
- Handles pipe-delimited `ending_reason` ('forced_error|other' ? 'forced_error')
- Output columns: `rally_number, start_time, end_time, ending_reason, sport, shots_count,
original_rally_number, notes`

During CVAT conversion: `/src/parsers/cvat/rally_cvat.py` lines 366-394
- Explicit fps resolution order: user-supplied `--fps` > ffprobe of `--video-path` > error
(never defaults to 30)
- Recomputes frame range ? seconds: `start_time = frame_min / fps`, `end_time = frame_max / fps`
- No configurable offset correction ? this is a TODO if the vendor's encoded video has a
leading padding

No per-batch offset correction currently implemented in the code. If Roora receives a re-
encoded copy with leading silence/padding, the returned timestamps will be offset relative to
the original. This is documented as a constraint: do not re-encode.

---
```

4. GOLDEN-SET BATCHES (GS-1..GS-4) ? WHAT WAS SENT & QUALITY NOTES

```

Pilot status (GS-1 candidate):
- File: `/data/validation/pilot_badminton_01_report.md`
- Scope: 1 badminton video (shorter clip, ~2?3 min estimated)
- What was sent: Unknown; intake manifest still has TODOs (INTAKE_MANIFEST_TEMPLATE.csv
lines 2-4)
- What came back: 24 rallies in CVAT XML export
- Format received: CVAT export (image or track mode) with box attributes (rally_number,
shots_count, ending_reason, etc.)

Roora pilot delivery quality issues (from pilot_badminton_01_report.md):
- Blocker (1): Rally 1 overlaps Rally 2 by 18.52 s (ends 21.02 s but Rally 2 starts at 2.50
s) ? invalid chronology, suggests overlapping/duplicate box annotations in CVAT
- Medium (3):
  - Rally 2: 2 shots over 30.0 s (0.07/s) ? implausibly low shot density, likely inflated span
from stray CVAT box
  - Rally 7: 2 shots over 16.0 s (0.12/s) ? same issue
  - Rally 16: 2 shots over 17.5 s (0.11/s) ? same issue
- Root cause: CVAT boxes not properly trimmed or sampled; coarse sampling (30-frame step at
```


59.94 fps ? 0.50 s per frame) mentioned in issues.txt

- ****Gate:**** BLOCKED pending rework

****GS-2, GS-3, GS-4:**** Not yet delivered (Phase 0 guideline complete, vendor bake-off deferred; Roora pilot in progress).

5. CONCRETE FILE PATHS & SCHEMAS

****Annotation spec:****

- ``/sports-data-collector/docs/annotation/ROORA_BRIEF.md`` ? vendor-facing spec, rally CSV schema, phasing
- ``/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md`` ? illustrated guide, per-sport rules, edge cases
- ``/sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv`` ? per-video fps/video_id template (to be filled)
- ``/sports-data-collector/docs/annotation/NDA_TEMPLATE.md`` ? vendor contract template

****Ingestion code:****

- ``/sports-data-collector/src/cli/curate.py`` lines 97-214 (``import_local`` command) ? CSV/JSON parsing, validation, DB insert
- ``/sports-data-collector/src/cli/curate.py`` lines 216-362 (``_parse_local_annotations``) ? CSV DictReader + BadmintonRally object construction
- ``/sports-data-collector/src/core/parsing.py`` lines 32-49 ? mm:ss:HH:MM:SS to seconds conversion
- ``/sports-data-collector/src/core/db.py`` lines 76-88 ? badminton_rallies table schema with (video_id, rally_number) PK
- ``/sports-data-collector/src/core/validator.py`` lines 29-57 ? chronological + no-overlap checks

****CVAT conversion (vendor alternative):****

- ``/sports-data-collector/src/parsers/cvat/rally_cvat.py`` lines 366-394 (``convert`` function) ? XML?records, fps resolution, issue detection
- ``/sports-data-collector/src/parsers/cvat/rally_cvat.py`` lines 283-363 (``detect_issues``) ? overlap, zero_duration, shot_density, possible_missed_rally checks
- ``/sports-data-collector/data/roora_badminton_issues.txt`` ? pilot delivery issue report (coarse sampling, blocker overlap)

****Pilot delivery:****

- ``/sports-data-collector/data/validation/pilot_badminton_01_report.md`` ? validation report, 24 rallies, 1 blocker, 3 medium issues

****Indexer bridge (golden-set architecture):****

- ``/badminton-highlight-indexer/docs/GOLDEN_SET_VENDORING.md`` ? vendor selection process, security controls, bake-off spec
- ``/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md`` ? GS-1..GS-5 PR breakdown, AnnotationProvider registry
- ``/badminton-highlight-indexer/docs/GOLDEN_SET_PHASE1_VIDEOS.md`` ? round-1 corpus spec (resolution/fps per sport, diversity matrix)

****Example manifest (output side):****

- ``/sports-data-collector/data/eval_export/manifest.json`` ? ``local_path`` + video_id mappings for indexer ingestion

Key files: [{"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\ROORA_BRIEF.md", "why": "Vendor-facing specification: rally CSV schema, timestamps (mm:ss?decimal seconds), format rules, non-recode constraint, intake manifest, phasing (3-video pilot)", "lines": "1-132"}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\ANNOTATION_GUIDE.md", "why": "Illustrated supplement: per-sport rules (badminton/pickleball/table-tennis, singles & doubles), edge cases, warm-up exclusion, boundary tolerance ($\pm 0.3s$)", "lines": "1-227"}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\cli\\curate.py", "why": "import_local command: CSV parsing (DictReader, case-folded headers), safe float parsing, validation, DB insert, (video_id, rally_number) join key", "lines": "97-214, 216-362"}, {"path": "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\core\\parsing.py", "why": "Timestamp parsing: mm:ss:HH:MM:SS?decimal seconds"}]

```

conversion, safe_float_required (raises on missing, critical
field)","lines":"32-49"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\core\\db.py","why":"Database schema: badminton_rallies table, PRIMARY KEY
(video_id, rally_number), columns (start_time, end_time as
REAL)","lines":"76-88"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\core\\validator.py","why":"Validation: no-overlap check (end[N] <= start[N+1]),
end_time > start_time, chronological by
rally_number","lines":"29-57"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\parsers\\cvat\\rally_cvat.py","why":"CVAT?CSV conversion: fps resolution
(explicit > ffprobe > error), frame?seconds recompute (seconds = frame/fps), issue detection
(overlap, shot_density, missed_rally)","lines":"1-35, 366-394,
283-363"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\data\\validation\\pilot_badminton_01_report.md","why":"Pilot delivery: 24 rallies, 1
blocker (overlap Rally 1?2), 3 medium (shot_density), coarse CVAT sampling issue (30 frames
~0.5s step)","lines":"1-16"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\annotation\\INTAKE_MANIFEST_TEMPLATE.csv","why":"Per-video metadata sent to
vendor: video_id, fps, duration, expected_rallies (template,
unfilled)","lines":"1-5"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\cli\\normalize_annotations.py","why":"Pre-import normalization: mm:ss?seconds,
drop invalid rows, renumber fractional/duplicate rally_numbers, output lean
schema","lines":"1-50, 77-189"},"path":"C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\PIPELINE_ROLE.md","why":"Bridge to indexer: video identity (collector uses
video_id, indexer uses MD5), manifest carries local_path, re-encode changes MD5 (no timestamp
offset handling)","lines":"40-45"},"path":"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_VENDORING.md","why":"Vendor selection & security: bake-off process,
Roora selected for initial pilot (verified Bangalore location), DPA controls, no user
uploads","lines":"1-65, 130-165"},"path":"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_IMPLEMENTATION.md","why":"Architecture: AnnotationProvider registry
(FileProvider, HumanVendorProvider, AutomatedProvider), GS-1..GS-5 PR breakdown, GS-3 parked on
Roora pilot","lines":"1-68"},"path":"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md","why":"Corpus spec: resolution/fps per sport
(badminton 30 ok/50-60 better, pickleball 30-60, table tennis 60 min), static-primary profile,
diversity matrix","lines":"38-67"]}
Quotes: ["C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\annotation\\R00RA_BRIEF.md:52
? 'Times are relative to the **start of the file we provide**. Do **not** trim, clip, or re-
encode the video ? our pipeline identifies a video by its file
checksum.'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\annotation\\R00RA_BRIEF.md:83-86 ? '**Boundaries:** `start_time`/`end_time`
within  $\pm 0.3$  s** of true serve-contact / landing. ... **Format:** decimal seconds present,
valid columns, non-overlapping, chronological.'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\annotation\\R00RA_BRIEF.md:51-54 ? '`start_time`/`end_time` that reach us
**must be decimal seconds** (the formula guarantees this). ... No overlaps: `end_time[N] <=
start_time[N+1]`.'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\cli\\curate.py:302-303 ? 'rally_number=_safe_int(row.get(`rally_number`)), idx
+ 1), start_time=_safe_float(row.get(`start_time`)),
end_time=_safe_float(row.get(`end_time`)), ...'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\core\\db.py:86 ? 'PRIMARY KEY (video_id, rally_number), FOREIGN KEY (video_id)
REFERENCES videos(video_id) ON DELETE CASCADE'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\parsers\\cvat\\rally_cvat.py:23-24 ? 'A CVAT job that samples every `step`
frames and assumes 30 fps yields systematically wrong times ... The frame index is reliable; the
vendor's seconds are not.'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\src\\parsers\\cvat\\rally_cvat.py:266-267 ? 'start_time=fmin / fps, end_time=fmax /
fps, ...'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\data\\validation\\pilot_badminton_01_report.md:8-9 ? '[BLOCKER] - (overlap) Rally 1
(ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s ? rallies must be
disjoint.'","C:\\Users\\avidu\\Projects\\sports-data-
collector\\data\\roora_badminton_issues.txt:2-3 ? 'fps used : 59.940 ... sampling step : 30
frames (~0.50s) ... [BLOCKER] - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s)
by 18.52s'","C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_VENDORING.md:152 ? 'Roora (Roora ML Pvt Ltd) | **Bengaluru** ? HSR
Layout (verified, `rooragr.com`) | ... **Selected for the initial paid pilot (2026-06-02)** ?
existence/capability/location verified; **quality TBD via the
pilot**.'","C:\\Users\\avidu\\Projects\\sports-data-collector\\docs\\PIPELINE_ROLE.md:42-45 ?
'The indexer keys videos by **file MD5**, this repo by a human `video_id`. A re-encode/re-
download changes the bytes, hence the MD5. So the manifest carries the exact `local_path`, and

```

any downstream processing must run on **that** file.'" , "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md:44-46 ? 'Badminton | 1080p (720p floor) | 30 ok, **50/60 better** | Shuttle decelerates fast; 60 sharpens smashes. Pickleball | 1080p | **30?60** | Ball slower than TT; 30 likely fine, 60 safer. Table tennis | 1080p | **60 min (120 ideal)**'"]

Your job: re-open the cited files and CHECK the load-bearing claims. Specifically verify:

1. The claimed delay numbers/direction for Roora annotations (quote the actual doc text).
2. The claimed annotation->video join key (filename vs MD5 video_id) in the ingestion code ? read the actual parser/ingestion code to confirm.
3. The claimed timestamp unit and precision of vendor annotations.
4. Whether any existing ffmpeg helper already does downscale/CFR/keyframe-dense re-encode. Flag anything the researchers got wrong, overstated, or that doesn't exist at the cited location. Also flag any CRITICAL fact they missed that changes the recommendation (e.g. an existing offset-correction mechanism, or a doc stating Roora's tool constraints).

2. user (2026-06-12T09:07:18.874Z)

6 June 2026

Collect

Illustrated Annotation Guide ? Rally Labeling (Supplement).pdf

Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf

Roora Pilot Review ? Badminton (2026-06-06).md

Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md

golden_labels_badminton.csv

rally_review.csv

3. user (2026-06-12T09:07:20.586Z)

C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf

C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Badminton (2026-06-06).md

C:\\Users\\avidu\\Projects\\Annotation Setup\\6 June 2026\\Feedback\\Roora Pilot Review - Badminton (2026-06-06).pdf

C:\\Users\\avidu\\Projects\\Annotation Setup\\Roora Pilot Review ? Pickleball & Table Tennis (2026-06-10).md

C:\\Users\\avidu\\Projects\\Annotation Setup\\6 June 2026\\Feedback\\Roora Pilot Review - Pickleball and Table Tennis (2026-06-10).pdf

4. user (2026-06-12T09:07:28.919Z)

1 # Roora Pilot Review ? Pickleball & Table Tennis

2

3 **Date:** 10 June 2026

4 **Re:** Phase-A calibration pilot ? pickleball + table tennis deliveries (CVAT exports; `Video 2 - Pickleball.mp4`, `Video 3 - Tabletennis.mp4`)

5 **Companion docs:** *Roora Annotation Brief ? Rally Golden Set (v2)*, *Illustrated Annotation Guide (v2)*, *Pilot Review ? Badminton (2026-06-06)*

6

7 ---

8

9 ## 1. Summary

10

11 Thank you for completing the pilot trio ? both deliveries ingest cleanly, shot counts are filled and plausible, and numbering and rally separation are clean. However, this round has **one serious issue: rally boundary accuracy.** Every rally's start/end in both videos is off by **0.5 s or more ? up to ~2 s in several** ? against the agreed tolerance of **±0.3 s** (Brief §4, Guide Part 3). The badminton round's timings were generally good, so this is a regression; we'd like to understand what changed in the process. The sections below explain the impact, the root cause we found in the files, and the conventions to lock before the Phase-B batches.

12

13 ## 2. Boundary accuracy ? outside the agreed tolerance *(most important)*

14

15 **What we found.** A full frame-by-frame re-check of all 12 rallies shows every

`start`/`end` boundary off by 0.5 s or more, several by up to ~2 s.

16

17 ****Root cause (from your files).**** All 24 rally boundaries sit ****exactly on the 30-frame sampling grid**** ? at 59.94 fps that grid is ~0.5 s apart, so a boundary placed only on grid frames ***cannot*** meet ± 0.3 s. Sampling every 30th frame ****inside**** a rally is fine; but the ****first box must be on the exact serve-contact frame and the last box on the exact ball-dead frame**** (Guide Part 3), even when those fall between grid steps.

18

19 ****Why this matters so much.**** These labels are the golden ground truth we train and evaluate a rally-segmentation model against ? ****boundary precision is the product itself****. The rallies in these videos run 178.5 s, so a 0.5?2 s boundary error corrupts 10?100% of a rally's span; the 1.0 s table-tennis rally #2 can be erased entirely by it. Concretely: at our standard evaluation gate (overlap ? 50%), a typical 4.5 s rally labeled 2 s late ****fails to match even a perfectly correct detection**** ? the dataset would score correct model output as wrong and wrong output as right. Imprecise gold is worse than no data, because it silently corrupts every model trained on it and every quality number reported from it.

20

21 ****Going forward:**** place the first/last box of every rally on the exact contact/dead frame (frame-step to find it ? Guide Parts 3 and 6), and self-check boundaries against ± 0.3 s before submitting (Guide Part 7 checklist).

22

23 **## 3. Rallies already in progress when the video starts ? skip them**

24

25 Pickleball ****rally #1 starts at the very first frame of the file (0:00.000)****. The serve contact is not in the video ? the file begins mid-rally ? so its start time is unknowable and the rally should not have been labeled at all.

26

27 ****Convention to follow from now on:**** if a video begins during an ongoing rally, ****skip that rally entirely and start annotating from the first rally that starts (serve contact visible) within the video.**** Never extrapolate or guess a boundary you did not see. The same applies at the end of a file: if the video cuts off mid-rally, skip that rally. When in doubt, flag it to us rather than guessing (Guide Part 6).

28

29 **## 4. Reminder for Phase B ? dead time and warm-up**

30

31 Phase-B videos are ~30 minutes and will contain substantial non-match footage, so we want to re-emphasize two rules now (Guide Rule 1 + Part 6):

32

33 - ****Skip all dead time**** ? before the match, between rallies, during breaks: walking, picking up the ball, serve preparation, changeovers, coaching. None of it is ever labeled.

34 - ****Skip warm-up / knock-up rallies and practice points**** ? they are not real rallies. Labeling starts at the ****first real, scored serve****. When in doubt whether the match has started, flag it to us.

35

36 **## 5. Schema hygiene ? remove the fields we did not ask for**

37

38 Please remove the following from the CVAT label schema; they add noise and, in one case, actively confusing data:

39

40 - ****`start time` / `end time`**** ? the delivered values are decimal minutes computed at an assumed 15 fps, ~4x off from real time (the videos are 59.94 fps). We derive all timing from the box frame ranges, so these fields serve no purpose and can only mislead. Remove them.

41 - ****`score\_before(A-B)` / `score\_after(A-B)`**** ? empty on every box in both videos. We are dropping scores from the schema; remove them.

42

43 The schema going forward is exactly: ****`rally\_number`, `shot\_count`, `ending\_reason`, `last\_shot\_outcome`, `note`**** ? nothing else.

44

45 **## 6. Field quality**

46

47 - ****`last\_shot\_outcome`**** ? pickleball has the literal value `0` on all 6 rallies (not a valid value); table tennis left it blank. The vocabulary is `in | net | out | n\_a` ? please use it.

48 - ****`ending\_reason` = `other` is overused**** ? 5 of 6 table-tennis rallies. `other` is the ***last resort*** (Guide Part 4): "out of table" and "net" endings map to

```

`forced_error`/`unforced_error` (with `last_shot_outcome` = `out`/`net`). Also, a note is
**required** whenever `other` is used ? 3 of the 5 `other` rallies have none.
49 - **One value per rally** ? table-tennis rally #5 carries `other` on some frames and
`winner` on others. Attribute values must be identical on every box of a rally.
50
51 ## 7. What was good ? please keep doing it
52
53 - In-rally sampling discipline is excellent: boxes every 30 frames with zero gaps in all
12 rallies.
54 - Clean sequential numbering, no overlapping rallies.
55 - `shot_count` filled everywhere and plausible against rally durations (including the
fast 12-shot table-tennis rally).
56 - Both sports delivered together, in a format our ingestion reads directly.
57
58 ## 8. Before Phase B
59
60 - We will fold the conventions above (exact first/last frames, skip rallies in progress
at file start/end, dead-time and warm-up rules, the reduced field schema) into the next revision
of the Guide ? this review is the joint calibration loop the Brief (§3, Phase A) describes.
61 - Please confirm which version of the Brief/Guide your team worked from on these two
videos.
62

```

5. user (2026-06-12T09:07:29.430Z)

```

1 # Roora Pilot ? Badminton Review & Conventions for Scale-up
2
3 **Date:** 6 June 2026
4 **Re:** Phase-A calibration pilot ? badminton job (CVAT job id 4075945)
5 **Companion docs:** *Roora Annotation Brief ? Rally Golden Set (v2)*, *Illustrated
Annotation Guide*
6
7 ---
8
9 ## 1. Summary
10
11 Thank you ? the first badminton delivery is a solid calibration round. We've built an
ingestion
12 adapter on our side, so **your CVAT export works for us directly** and the timing/format
mechanics
13 are handled here, not in your workflow. After a full frame-by-frame review, the rally
**timings
14 are generally good** ; the items that genuinely need your attention are about
**annotation
15 completeness and rally boundaries** ? and they matter most for the larger batches coming
next.
16
17 ---
18
19 ## 2. Handled on our side ? no change needed from you
20
21 - **Export format.** Keep working in **CVAT** ; we ingest your bounding-box export as-is.
No need
22 to produce a CSV.
23 - **Timestamps.** We recompute each rally's time from the box **frame range at the true
fps**
24 (`seconds = frame / fps`), so the CVAT time attributes don't matter, and `mm:ss` vs
decimal is
25 a non-issue ? our ingestion parses both. Minor boundary slack is absorbed by padding
downstream.
26 - **Per-frame label drift.** Where a rally's boxes carried slightly different attribute
values
27 across frames, we collapse to one value per rally automatically.
28
29 ## 3. What needs your attention (for the upcoming larger batches)

```

```

30
31     These can't be fixed in code ? they're annotation-quality conventions.
32
33     ### 3.1 Completeness ? don't miss rallies *(most important)*
34     Our review found rallies with **no label**:
35     - **~1:46** ? a 5-shot rally omitted entirely (frame grab attached).
36     - **3:34?3:43** ? a full rally omitted because it was **absorbed into the previous
rally's box**
37     (the box ran from 3:25 all the way to 3:43, covering two rallies + the dead time
between).
38
39     The brief calls a missed rally *"the most serious error."* For 30-min videos this is the
#1 thing
40     to get right.
41
42     ### 3.2 One rally = one continuous interval
43     A few boxes spanned **past the shuttle-dead frame** or **across two rallies**:
44
45     | rally | delivered span | true rally | issue |
46     |-----|-----|-----|-----|
47     | 2 | 0:02?0:32 | 0:29?0:32 | box started ~27 s into dead time |
48     | 7 | 1:35?1:51 | 1:35?1:38 | box ran ~13 s past the end |
49     | 16 | 3:25?3:43 | 3:25?3:29 | box covered a **second** rally (see 3.1) |
50
51     Convention: start the box at the **serve-contact** frame, end it at the **shuttle-dead**
frame,
52     and start a **new** box for the next rally ? never let one box bridge a gap.
53
54     ### 3.3 Endings ? `winner` vs an error
55     A `winner` means the opponent made **no legal contact**. A few rallies where the shuttle
landed
56     **out** were marked `winner` (e.g. 6, 7, 9). If the rally ended on a miss, it's a
57     `forced_error`/`unforced_error`, not a winner.
58
59     ### 3.4 New field ? `last_shot_outcome` *(small label-schema add)*
60     We're adding one attribute so the physical outcome is captured alongside the reason:
61
62     | field | values |
63     |-----|-----|
64     | `ending_reason` | winner \| forced_error \| unforced_error \| service_fault \| let \|
other |
65     | `last_shot_outcome` | in \| net \| out \| n_a |
66
67     Just add `last_shot_outcome` to the CVAT label schema (one dropdown). If that's awkward,
tell us
68     and we'll keep it optional.
69
70     ### 3.5 Scope ? pickleball & table tennis
71     The pilot is 1 badminton + 1 pickleball + 1 table tennis; we've received badminton only.
Please
72     confirm status / ETA for the other two.
73
74     ---
75
76     ## 4. Convention checklist (for the next batches)
77
78     - [ ] **Every** served point labeled ? no missed rallies, including back-to-back ones.
79     - [ ] One box per rally: serve-contact ? shuttle-dead; never bridge a gap or two
rallies.
80     - [ ] `ending_reason` ? {winner, forced_error, unforced_error, service_fault, let,
other}.
81     - [ ] `last_shot_outcome` ? {in, net, out, n_a} (one dropdown).
82     - [ ] `shots_count` = total racket contacts (serve = 1).
83     - [ ] CVAT export is fine ? we handle the rest.
84

```



```

136         # 1. Parse Annotations
137         annotations = []
138         try:
139             annotations = self._parse_local_annotations(sport_str, video_id,
140 annotations_path)
141         except Exception as e:
142             print(f"Error parsing annotations: {e}")
143             sys.exit(1)
144
145         # 2. Run Validation
146         is_valid = True
147         errors = []
148         if sport_str == "badminton":
149             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
150         elif sport_str == "tennis":
151             is_valid, errors = self.validator.validate_tennis_points(annotations)
152         elif sport_str == "cricket":
153             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
154         elif sport_str == "pickleball":
155             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
156         elif sport_str == "tabletennis":
157             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
158
159         if not is_valid:
160             print(f"Error: Annotations failed validation!")
161             for err in errors:
162                 print(f" - {err}")
163             sys.exit(1)
164
165         # 2b. Pre-insert safety ? checked BEFORE any DB write, so a rejection never
166         # leaves a half-updated DB (a committed video row with rolled-back annotations).
167         self._guard_import(sport_str, video_id, annotations, force=getattr(args,
168 "force", False))
169
170         # 3. Create Video Metadata
171         video_meta = VideoMetadata(
172             video_id=video_id,
173             sport=SportType(sport_str),
174             video_url=None,
175             local_path=args.video_path,
176             license_type=license_type,
177             license_url=None,
178             is_commercial=is_commercial,
179             status=CurationStatus.CURATED, # Custom local imports bypass staging by
180 default
181             match_name=match_name,
182             fps=args.fps,
183             resolution=args.resolution
184         )
185
186         # 4. Insert into Database
187         self.db.insert_video(video_meta)
188
189         count = 0
190         if sport_str == "badminton":
191             count = self.db.insert_badminton_rallies(annotations)
192         elif sport_str == "tennis":
193             count = self.db.insert_tennis_points(annotations)
194         elif sport_str == "cricket":
195             count = self.db.insert_cricket_deliveries(annotations)
196         elif sport_str == "pickleball":
197             count = self.db.insert_pickleball_rallies(annotations)
198         elif sport_str == "tabletennis":
199             count = self.db.insert_tabletennis_rallies(annotations)
200

```



```

260         start_time=_safe_float(obj.get("start_time")),
261         end_time=_safe_float(obj.get("end_time")),
262         batsman=obj.get("batsman"),
263         bowler=obj.get("bowler"),
264         runs=_safe_int(obj.get("runs"), None),
265         wicket=bool(obj.get("wicket", False)),
266         extras=_safe_int(obj.get("extras"), 0)
267     ))
268 elif sport == "pickleball":
269     parsed_items.append(PickleballRally(
270         video_id=video_id,
271         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
272         start_time=_safe_float(obj.get("start_time")),
273         end_time=_safe_float(obj.get("end_time")),
274         score_before=obj.get("score_before"),
275         score_after=obj.get("score_after"),
276         shots_count=_safe_int(obj.get("shots_count"), None),
277         ending_reason=obj.get("ending_reason"),
278         last_shot_outcome=obj.get("last_shot_outcome")
279     ))
280 elif sport == "tabletennis":
281     parsed_items.append(TableTennisRally(
282         video_id=video_id,
283         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
284         start_time=_safe_float(obj.get("start_time")),
285         end_time=_safe_float(obj.get("end_time")),
286         score_before=obj.get("score_before"),
287         score_after=obj.get("score_after"),
288         shots_count=_safe_int(obj.get("shots_count"), None),
289         ending_reason=obj.get("ending_reason"),
290         last_shot_outcome=obj.get("last_shot_outcome")
291     ))
292 elif ext == '.csv':
293     with open(filepath, 'r', encoding='utf-8') as f:
294         reader = csv.DictReader(f)
295         # Normalize headers
296         reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
297
298     for idx, row in enumerate(reader):
299         if sport == "badminton":
300             parsed_items.append(BadmintonRally(
301                 video_id=video_id,
302                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
303                 start_time=_safe_float(row.get("start_time")),
304                 end_time=_safe_float(row.get("end_time")),
305                 score_before=row.get("score_before"),
306                 score_after=row.get("score_after"),
307                 shots_count=_safe_int(row.get("shots_count"), None),
308                 ending_reason=row.get("ending_reason"),
309                 last_shot_outcome=row.get("last_shot_outcome")
310             ))
311         elif sport == "tennis":
312             parsed_items.append(TennisPoint(
313                 video_id=video_id,
314                 point_number=_safe_int(row.get("point_number"), idx + 1),
315                 start_time=_safe_float(row.get("start_time")),
316                 end_time=_safe_float(row.get("end_time")),
317                 set_score=row.get("set_score"),
318                 game_score=row.get("game_score"),
319                 server=row.get("server"),
320                 ending_type=row.get("ending_type")
321             ))
322         elif sport == "cricket":
323             parsed_items.append(CricketDelivery(

```

```

324         video_id=video_id,
325         over=_safe_int(row.get("over"), 0),
326         ball=_safe_int(row.get("ball"), 0),
327         start_time=_safe_float(row.get("start_time")),
328         end_time=_safe_float(row.get("end_time")),
329         batsman=row.get("batsman"),
330         bowler=row.get("bowler"),
331         runs=_safe_int(row.get("runs"), None),
332         wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
333         extras=_safe_int(row.get("extras"), 0)
334     ))
335     elif sport == "pickleball":
336         parsed_items.append(PickleballRally(
337             video_id=video_id,
338             rally_number=_safe_int(row.get("rally_number"), idx + 1),
339             start_time=_safe_float(row.get("start_time")),
340             end_time=_safe_float(row.get("end_time")),
341             score_before=row.get("score_before"),
342             score_after=row.get("score_after"),
343             shots_count=_safe_int(row.get("shots_count"), None),
344             ending_reason=row.get("ending_reason"),
345             last_shot_outcome=row.get("last_shot_outcome")
346         ))
347     elif sport == "tabletennis":
348         parsed_items.append(TableTennisRally(
349             video_id=video_id,
350             rally_number=_safe_int(row.get("rally_number"), idx + 1),
351             start_time=_safe_float(row.get("start_time")),
352             end_time=_safe_float(row.get("end_time")),
353             score_before=row.get("score_before"),
354             score_after=row.get("score_after"),
355             shots_count=_safe_int(row.get("shots_count"), None),
356             ending_reason=row.get("ending_reason"),
357             last_shot_outcome=row.get("last_shot_outcome")
358         ))
359     else:
360         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
361
362     return parsed_items
363
364 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
365     """Download one video at best resolution, retrying transient failures.
366
367     A below-min-height result almost always means YouTube rate-limited the
368     request and served a low fallback (not that the video lacks HD), so we
369     retry with a growing backoff. Returns (DownloadResult, None) on success,

```

8. user (2026-06-12T09:07:50.339Z)

```

60         sport TEXT NOT NULL,
61         video_url TEXT,
62         local_path TEXT,
63         license_type TEXT NOT NULL,
64         license_url TEXT,
65         annotation_license TEXT,
66         is_commercial INTEGER NOT NULL DEFAULT 0,
67         status TEXT NOT NULL DEFAULT 'staging',
68         match_name TEXT NOT NULL,
69         fps REAL,
70         resolution TEXT
71     )
72     """)
73

```

```

74         # 2. Create badminton_rallies table
75         cursor.execute("""
76         CREATE TABLE IF NOT EXISTS badminton_rallies (
77             video_id TEXT NOT NULL,
78             rally_number INTEGER NOT NULL,
79             start_time REAL NOT NULL,
80             end_time REAL NOT NULL,
81             score_before TEXT,
82             score_after TEXT,
83             shots_count INTEGER,
84             ending_reason TEXT,
85             last_shot_outcome TEXT,
86             PRIMARY KEY (video_id, rally_number),
87             FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
88         )
89         """)
90
91         # 3. Create tennis_points table
92         cursor.execute("""
93         CREATE TABLE IF NOT EXISTS tennis_points (
94             video_id TEXT NOT NULL,
95             point_number INTEGER NOT NULL,
96             start_time REAL NOT NULL,
97             end_time REAL NOT NULL,
98             set_score TEXT,
99             game_score TEXT,
100            server TEXT,
101            ending_type TEXT,
102            PRIMARY KEY (video_id, point_number),
103            FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
104        )
105        """)
106
107        # 4. Create cricket_deliveries table
108        cursor.execute("""
109        CREATE TABLE IF NOT EXISTS cricket_deliveries (
110            video_id TEXT NOT NULL,
111            over_num INTEGER NOT NULL,
112            ball_num INTEGER NOT NULL,
113            start_time REAL NOT NULL,
114            end_time REAL NOT NULL,
115            batsman TEXT,
116            bowler TEXT,
117            runs INTEGER,
118            wicket INTEGER DEFAULT 0,
119            extras INTEGER DEFAULT 0,

```

9. user (2026-06-12T09:07:59.684Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default

```

```

16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24         if value is None:
25             raise ValueError("missing required numeric value")
26         s = str(value).strip()
27         if not s:
28             raise ValueError("missing required numeric value")
29         return float(s)
30
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34         if not time_str:
35             return 0.0
36         s = str(time_str).strip()
37         if ":" in s:
38             parts = s.split(":")
39             try:
40                 if len(parts) == 3:
41                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                 if len(parts) == 2:
43                     return float(parts[0]) * 60 + float(parts[1])
44             except ValueError:
45                 return 0.0
46         try:
47             return float(s)
48         except ValueError:
49             return 0.0
50

```

10. user (2026-06-12T09:08:00.404Z)

```

20         'commercial_licenses',
21         ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
22     )
23
24     def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
25         """Determines if a license type is safe for commercial use based on the
configuration whitelist."""
26         val = license_type.value if isinstance(license_type, LicenseType) else
license_type
27         return val in self.commercial_licenses
28
29     def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
30         """
31         Validates badminton rallies:
32         - Sorted chronologically.
33         - Start time < End time (handled by Pydantic, but double checked).
34         - No temporal overlap between sequential rallies.
35         """
36         errors = []
37         if not rallies:
38             return True, errors
39
40         # Sort by rally number to check progression

```

```

41         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
42
43         for i in range(len(sorted_rallies)):
44             r = sorted_rallies[i]
45             if r.start_time >= r.end_time:
46                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47
48             if i > 0:
49                 prev = sorted_rallies[i - 1]
50                 if r.start_time < prev.end_time:
51                     errors.append(
52                         f"Rally overlap detected between Rally {prev.rally_number} "
53                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                         f"(starts at {r.start_time}s)"
55                     )
56
57         return len(errors) == 0, errors
58
59     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60         errors = []
61         if not points:
62             return True, errors
63
64         sorted_pts = sorted(points, key=lambda p: p.point_number)
65         for i in range(len(sorted_pts)):
66             pt = sorted_pts[i]
67             if pt.start_time >= pt.end_time:
68                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69

```

11. user (2026-06-12T09:08:07.651Z)

```

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:11:from src.core.parsing import
safe_int as _safe_int, safe_float_required as _safe_float
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:236:
start_time=_safe_float(obj.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:237:
end_time=_safe_float(obj.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:248:
start_time=_safe_float(obj.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:249:
end_time=_safe_float(obj.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:260:
start_time=_safe_float(obj.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:261:
end_time=_safe_float(obj.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:272:
start_time=_safe_float(obj.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:273:
end_time=_safe_float(obj.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:284:
start_time=_safe_float(obj.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:285:
end_time=_safe_float(obj.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:303:
start_time=_safe_float(row.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:304:
end_time=_safe_float(row.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:315:
start_time=_safe_float(row.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:316:
end_time=_safe_float(row.get("end_time")),

```

```

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:327:
start_time=_safe_float(row.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:328:
end_time=_safe_float(row.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:339:
start_time=_safe_float(row.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:340:
end_time=_safe_float(row.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:351:
start_time=_safe_float(row.get("start_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:352:
end_time=_safe_float(row.get("end_time")),
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:9:      a
blank/missing time makes ``safe_float_required`` raise; a reversed
C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py:3:Previously these were
duplicated as private ``_safe_int`` / ``_safe_float`` /
C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py:4:``_parse_time_string`` in
both ``src/cli/curate.py`` and the CoachAI parser.
C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py:22:def
safe_float_required(value: Any) -> float:
C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py:32:def
parse_time_string(time_str: Any) -> float:
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py:5:from
src.core.parsing import safe_int as _safe_int, parse_time_string
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py:83:
time_val = parse_time_string(raw_time)

```

12. assistant (2026-06-12T09:08:16.836Z)

``\_safe\_float`` is aliased to ``safe\_float\_required`` ? confirms that part. Now the CVAT converter and the vendor brief/guide.

13. user (2026-06-12T09:08:17.785Z)

```

1      """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3      Why this exists
4      -----
5      Our annotation brief (``docs/annotation/ROORA_BRIEF.md``) asks vendors for a
6      per-rally CSV that imports directly via ``curate import-local``. A vendor may
7      instead do the work in CVAT and hand back the CVAT export: per-frame bounding
8      boxes, with the rally fields (``rally_number``, ``shots_count``,
9      ``ending_reason`` ...) carried as CVAT object attributes on each box. The box
10     geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11     metadata to.
12
13     This module is the ingestion-side adapter that absorbs that vendor format so the
14     modeling repo never has to. It does two things, both deliberately on our side
15     so we fix mechanical problems rather than renegotiating them:
16
17     1. Recompute time from the frame range. A rally's interval is taken from the
18     first/last frame a box for that rally appears on:
19     ``seconds = frame / fps`` at the video's true fps. We ignore any
20     ``start_time``/``end_time`` attributes the vendor wrote ? a CVAT job that
21     samples every ``step`` frames and assumes 30 fps yields systematically wrong
22     times (we have seen ``start_time == frame / (step * 30)``, i.e. off by the
23     sampling step at the wrong fps). The frame index is reliable; the vendor's
24     seconds are not.
25
26     2. Surface ? never silently fix ? content defects. Overlapping rallies,
27     zero-duration rallies, ending_reason values outside the agreed set, and
28     too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29     rally boundary would poison ground truth, so that stays a human/vendor call.
30
31     Supported input: the single-file CVAT XML export, in either image mode

```

```

32     (``<image><box>...``) or **track/interpolation mode** (``<track><box frame=...>``).
33     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34     later if a vendor sends only those.
35     """
36     from __future__ import annotations
37
38     import os
39     import re
40     import zipfile
41     import xml.etree.ElementTree as ET
42     from collections import Counter
43     from dataclasses import dataclass, field
44     from typing import Any, Dict, List, Optional, Tuple
45
46     from src.core.parsing import safe_int
47     from src.core.downloader import probe_resolution_fps
48
49     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
50     # Kept here as a soft check only (a warning, never a hard failure) so this
51     # adapter does not depend on the annotation-spec wording.
52     DEFAULT_ALLOWED_REASONS = {
53         "winner",
54         "forced_error",
55         "unforced_error",
56         "service_fault",
57         "let",
58         "other",
59     }
60

```

14. user (2026-06-12T09:08:18.528Z)

```

240     gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
241     if not gaps:
242         return 1
243     return Counter(gaps).most_common(1)[0][0]
244
245
246     def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
247         """Group boxes by rally_number and derive each rally's frame range -> seconds."""
248         if fps is None or fps <= 0:
249             raise ValueError("fps must be a positive number to convert frames to seconds")
250
251         by_rally: Dict[int, List[Dict[str, Any]]] = {}
252         for b in boxes:
253             rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
254             if rn is None:
255                 continue
256             by_rally.setdefault(rn, []).append(b)
257
258         records: List[RallyRecord] = []
259         for rn in sorted(by_rally):
260             items = by_rally[rn]
261             frames = sorted(b["frame"] for b in items)
262             attrs = items[0]["attrs"]
263             fmin, fmax = frames[0], frames[-1]
264             records.append(RallyRecord(
265                 rally_number=rn,
266                 start_time=fmin / fps,
267                 end_time=fmax / fps,
268                 shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
269                 ending_reason=_attr_lookup(attrs, "ending_reason"),
270                 last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
271                 score_before=_attr_lookup(attrs, "score_before"),
272                 score_after=_attr_lookup(attrs, "score_after"),

```



```

273         notes=_attr_lookup(attrs, "notes"),
274         frame_min=fmin,
275         frame_max=fmax,
276         n_boxes=len(frames),
277         vendor_start_time=attrs.get("start_time"),
278         vendor_end_time=attrs.get("end_time"),
279     ))
280     return records
281
282
283     def detect_issues(records: List[RallyRecord],
284                     stop_frame: Optional[int] = None,
285                     allowed_reasons: Optional[set] = None,
286                     allowed_outcomes: Optional[set] = None,
287                     max_gap_s: float = 15.0) -> List[Issue]:
288         """Find the defects that block or degrade a clean ground-truth import.
289
290         Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
291         slack of ~0.5s is acceptable for this pipeline because the downstream
292         highlight compiler pads every clip on output (badminton-highlight-indexer,
293         backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
294         absorbs the sampling uncertainty when highlights are trimmed.
295         """
296         allowed = allowed_reasons if allowed_reasons is not None else
297         DEFAULT_ALLOWED_REASONS
298         allowed_out = allowed_outcomes if allowed_outcomes is not None else
299         DEFAULT_ALLOWED_OUTCOMES
300         issues: List[Issue] = []
301
302         ordered = sorted(records, key=lambda r: r.rally_number)
303         for i, r in enumerate(ordered):
304             if r.end_time <= r.start_time:
305                 issues.append(Issue(
306                     "blocker", "zero_duration",
307                     f"Rally {r.rally_number}: end <= start "
308                     f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",
309                     r.rally_number))
310
311             if r.ending_reason is not None and r.ending_reason not in allowed:
312                 issues.append(Issue(
313                     "medium", "ending_reason_vocab",
314                     f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
315                     f"not in the agreed set {sorted(allowed)}.",
316                     r.rally_number))
317
318             if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
319                 issues.append(Issue(
320                     "low", "outcome_vocab",
321                     f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
322                     f"not in {sorted(allowed_out)}.",
323                     r.rally_number))
324
325             # Inflated span: a long rally with implausibly few shots usually means a
326             # stray/duplicated box stretched the frame range past the real ending.
327             if r.shots_count and r.duration >= 5.0:
328                 sps = r.shots_count / r.duration
329                 if sps < 0.3:
330                     issues.append(Issue(
331                         "medium", "shot_density",
332                         f"Rally {r.rally_number}: {r.shots_count} shots over "
333                         f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
334                         f"may be inflated by a stray box.",
335                         r.rally_number))
336
337             if stop_frame is not None and r.frame_max > stop_frame:

```

```

336         issues.append(Issue(
337             "high", "frame_out_of_range",
338             f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
339             f"last frame {stop_frame}.",
340             r.rally_number))
341
342     if i > 0:
343         prev = ordered[i - 1]
344         if r.start_time < prev.end_time:
345             issues.append(Issue(
346                 "blocker", "overlap",
347                 f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
348                 f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
349                 f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
350                 r.rally_number))
351         elif r.start_time - prev.end_time > max_gap_s:
352             # A long unlabeled stretch between two rallies is a cheap, label-only
353             # second opinion for the worst error (a missed rally). It won't catch a
354             # rally MERGED into its neighbour's span (shot_density/inflated catches
355             # that), but it does catch an omitted rally that left a visible hole.
356             gap = r.start_time - prev.end_time
357             issues.append(Issue(
358                 "medium", "possible_missed_rally",
359                 f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
360                 f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
361                 f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
gap.",
362                 r.rally_number))
363     return issues
364
365
366 def convert(xml_path: str,
367            fps: Optional[float] = None,
368            video_path: Optional[str] = None,
369            step: Optional[int] = None,
370            allowed_reasons: Optional[set] = None) -> ConversionResult:
371     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
372
373     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
374     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
375     that silent guess is exactly the bug this adapter exists to undo.
376     """
377     meta, boxes = parse_cvat_boxes(xml_path)
378
379     if fps is None and video_path:
380         _, _, probed = probe_resolution_fps(video_path)
381         fps = probed
382     if fps is None or fps <= 0:
383         raise ValueError(
384             "Could not determine fps. Pass fps=... or a readable video_path so "
385             "frames can be converted to seconds (seconds = frame / fps).")
386
387     if step is None:
388         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
389
390     records = build_records(boxes, fps)
391     issues = detect_issues(records,
392                           stop_frame=meta.get("stop_frame"),
393                           allowed_reasons=allowed_reasons)
394     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
meta=meta)
395
396
397 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
398     """Write records as the canonical per-rally CSV that ``import-local`` reads."""

```

```

399     import csv
400     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
401     with open(out_path, "w", newline="", encoding="utf-8") as f:
402         w = csv.writer(f)
403         w.writerow(CANONICAL_COLUMNS)
404         for r in sorted(records, key=lambda x: x.rally_number):
405             w.writerow([
406                 r.rally_number,
407                 _mmss(r.start_time), _mmss(r.end_time),
408                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
409                 r.shots_count if r.shots_count is not None else "",

```

15. assistant (2026-06-12T09:08:31.691Z)

Critical find in `detect\_issues` ? sampling slack is deliberately not flagged because downstream padding absorbs it. Now checking the padding config, ffmpeg helpers, and hashing.

16. user (2026-06-12T09:08:33.133Z)

```

60         "conda_env": "wasb",
61         "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62         "sport": "badminton",
63         "wsl_stage_dir": "~/clips",
64         "timeout_sec": 1800,
65         "velocity_threshold": 0.02
66     }
67 },
68 "output": {
69     "default_highlights_name": "highlights.mp4",
70     "db_name": "sports_indexer.db"
71 },
72 "stitching": {
73     "begin_padding": 0.5,
74     "end_padding": 0.5,
75     "use_cache": false,
76     "min_shot_count": 1,
77     "watermark": {
78         "enabled": true,
79         "text": "Generated by Khelsutra.guru",
80         "position": "bottom-right",
81         "opacity": 0.85,
82         "logo_path": "",
83         "font_path": ""
84     }
85 }
86 }

```

17. user (2026-06-12T09:08:33.546Z)

```

1     import subprocess
2     import logging
3     from typing import Optional
4
5     logger = logging.getLogger(__name__)
6
7     def get_video_duration(video_path: str) -> float:
8         """Returns the exact duration of a video file in seconds using ffprobe.
9         Returns 0.0 on failure."""
10        try:
11            duration_output = subprocess.check_output([
12                "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                "-of", "default=noprint_wrappers=1:nokey=1", video_path
14            ]).decode().strip()

```

```

15         return float(duration_output)
16     except Exception as e:
17         logger.error(f"Could not detect video duration for {video_path}: {e}")
18         return 0.0
19
20 def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                         output_path: str, reencode: bool = False,
22                         scale_width: Optional[int] = None) -> bool:
23     """Extracts a chunk from a video file.
24
25     Args:
26         reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                   If False, uses stream-copy (fast but cuts at nearest keyframe).
28                   Use True for short clips where keyframe drift matters.
29         scale_width: If set (requires reencode), downscale to this width (height auto,
30                     even). Useful for high-res sources headed to an API that analyses at
31                     low res anyway ? shrinks uploads with no analysis loss.
32     """
33     cmd = [
34         "ffmpeg", "-y",
35         "-ss", str(start_time),
36         "-t", str(duration),
37         "-i", video_path,
38     ]
39     if reencode:
40         if scale_width:
41             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43         else:
44             cmd += ["-c", "copy"]
45     cmd.append(output_path)
46     try:
47         subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
48 stderr=subprocess.DEVNULL)
49         return True
50     except Exception as e:
51         logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
52 {e}")
53         return False
54

```

18. user (2026-06-12T09:08:33.950Z)

```

1     """Canonical file-identity hashing for the indexer.
2
3     `video_id` is the MD5 of the *whole* video file. Historically each entriypoint
4     (`backend/main.py`, `run_eval.py`, `run_phase3_eval.py`, `run_process_and_stitch.py`)
5     defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6     The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7     any future change to one copy would silently diverge the DB key.
8
9     This module is the single source of truth. Import `compute_video_id` everywhere a
10    video needs to be keyed.
11    """
12
13    from __future__ import annotations
14
15    import hashlib
16
17    # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18    # was centralized. The chunk size does not affect the resulting digest ? it only
19    # bounds memory use while streaming large files.
20    _CHUNK_SIZE = 65536
21
22

```

```

23 def compute_video_id(filepath: str) -> str:
24     """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26     Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27     into memory at once.
28     """
29     hasher = hashlib.md5()
30     with open(filepath, "rb") as f:
31         for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32             hasher.update(chunk)
33     return hasher.hexdigest()
34

```

19. user (2026-06-12T09:08:41.642Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from typing import Any, Dict, List, Optional, Tuple, Union
6
7     logger = logging.getLogger(__name__)
8
9     class VideoCompiler:
10         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
11             0.5,
12                 watermark: Optional[Any] = None):
13             self.output_dir = output_dir
14             self.end_padding = end_padding
15             self.begin_padding = begin_padding
16             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
17             # normalized to a dict (or None) so this module stays config-package-agnostic.
18             self.watermark = self._normalize_watermark(watermark)
19             os.makedirs(output_dir, exist_ok=True)
20
21         @staticmethod
22         def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
23             if watermark is None:
24                 return None
25             if hasattr(watermark, "model_dump"):
26                 return watermark.model_dump()
27             if isinstance(watermark, dict):
28                 return dict(watermark)
29             return None
30
31         @staticmethod
32         def _ff_quote(path: str) -> str:
33             """Make a filesystem path safe to embed inside single quotes in an ffmpeg
34             filtergraph: forward slashes (Windows-friendly) + escaped single quotes."""
35             return path.replace("\\", "/").replace("'", r"\'")
36
37         @staticmethod
38         def _parse_time(val: Union[int, float, str]) -> float:
39             """Normalizes a timestamp value to float seconds.
40             Handles: float/int (pass-through), plain numeric strings ("12.4"),
41             and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
42             This mirrors the parse_time logic used in gemini.py to ensure
43             the stitcher can always consume whatever format the indexer produced."""
44             if isinstance(val, (int, float)):
45                 return float(val)
46             if isinstance(val, str):
47                 val = val.strip()
48                 if ":" in val:
49                     # Handle MM:SS, HH:MM:SS, etc.
50                     parts = val.split(":")

```

```

50         parts.reverse()
51         total = 0.0
52         for i, p in enumerate(parts):
53             try:
54                 total += float(p) * (60 ** i)
55             except ValueError:
56                 pass
57         return total
58     try:
59         return float(val)
60     except ValueError:
61         logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
62         return 0.0
63     logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
64     return 0.0
65
66     def _get_video_duration(self, video_path: str) -> float:
67         """Probes the video file to get its total duration in seconds.
68         Returns 0.0 if it cannot be determined."""
69         try:
70             result = subprocess.run(
71                 [
72                     "ffprobe", "-v", "error",
73                     "-show_entries", "format=duration",
74                     "-of", "default=noprint_wrappers=1:nokey=1",
75                     video_path
76                 ],
77                 capture_output=True, text=True, check=True
78             )
79             return float(result.stdout.strip())
80         except Exception as e:
81             logger.warning(f"Could not determine video duration via ffprobe: {e}")
82             return 0.0
83
84     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
85         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
86         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
87         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
88         because the mark is already baked into every clip identically."""
89         wm = self.watermark
90         if not wm or not wm.get("enabled"):
91             return None
92
93         text = (wm.get("text") or "").strip()
94         logo_path = (wm.get("logo_path") or "").strip()
95         if logo_path and not os.path.exists(logo_path):
96             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
97         logo_path = ""
98         if not text and not logo_path:
99             return None
100
101         margin = int(wm.get("margin", 24))
102         opacity = float(wm.get("opacity", 0.85))
103         position = str(wm.get("position", "bottom-right"))
104         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
105         dt_xy = {
106             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
107             "bottom-left": f"x={margin}:y=h-th-{margin}",
108             "top-right": f"x=w-tw-{margin}:y={margin}",
109             "top-left": f"x={margin}:y={margin}",
110         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")

```

```

111         ov_xy = {
112             "bottom-right": f"W-w-{{margin}}:H-h-{{margin}}",
113             "bottom-left": f"{{margin}}:H-h-{{margin}}",
114             "top-right": f"W-w-{{margin}}:{{margin}}",
115             "top-left": f"{{margin}}:{{margin}}",
116         }.get(position, f"W-w-{{margin}}:H-h-{{margin}}")
117
118         drawtext = ""
119         if text:
120             ratio = float(wm.get("font_size_ratio") or 0.035)
121             divisor = max(1, round(1.0 / ratio))
122             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
123             tf_path = os.path.join(tmp_dir, "watermark.txt")
124             with open(tf_path, "w", encoding="utf-8") as fh:
125                 fh.write(text)
126             font_path = (wm.get("font_path") or "").strip()
127             if font_path:
128                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
129             else:
130                 font_opt = f"font='{wm.get('font', 'Sans')}'"
131             box = ""
132             if wm.get("box", True):
133                 box = f":box=1:boxcolor=black@{{min(opacity, 0.5)}}.2f}:boxborderw=10"
134             drawtext = (
135                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
136                 f":fontcolor=white@{{opacity:.2f}}:fontsize=h/{{divisor}}:{{dt_xy}}{box}"
137             )
138
139             if logo_path and drawtext:
140                 return
141             f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
142             if logo_path:
143                 return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
144             return drawtext
145
146         def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
147             """
148             Extracts selected segments from a raw video and stitches them together
149             into a seamless highlights file, applying end_padding to prevent abrupt endings.
150             """
151             if not segments:
152                 raise ValueError("No highlight segments provided to compile.")
153
154             if not os.path.exists(raw_video_path):
155                 raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
156
157             # Defense-in-depth: never let the output name escape output_dir (the API also
158             # validates this at the request boundary).
159             output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
160
161             # Probe video duration so we can detect out-of-range segments
162             video_duration = self._get_video_duration(raw_video_path)
163             if video_duration > 0:
164                 logger.info(f"Source video duration: {video_duration:.2f}s")
165             else:
166                 logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
167
168             # We will create temporary clips and stitch them using FFmpeg concat demuxer
169             temp_clips = []
170             skipped_segments = []

```

```

171         # Create a temp directory within output_dir
172         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
173             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
174             # Build the (optional) watermark filtergraph once; it is applied during
every
175             # per-clip re-encode so the mark is baked into each clip identically.
176             watermark_filter = self._watermark_filter(tmp_dir)
177             if watermark_filter:
178                 logger.info("[watermark] applying branding overlay to each clip.")
179             for idx, (raw_start, raw_end) in enumerate(segments):
180                 # Normalize timestamps to float seconds ? handles float, int,
181                 # plain numeric strings, and colon-separated formats like "MM:SS"
182                 start = self._parse_time(raw_start)
183                 end = self._parse_time(raw_end)
184                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
185
186                 # Validate the segment times
187                 if start < 0:
188                     logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
189                     start = 0.0
190
191                 if start >= end:
192                     logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
193                     skipped_segments.append((idx, start, end, "start >= end"))
194                     continue
195
196                 # Check if segment extends beyond video duration
197                 if video_duration > 0:
198                     if start >= video_duration:
199                         logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
200                                     f"This segment cannot be extracted ? the video ran
out. Skipping.")
201                         skipped_segments.append((idx, start, end, "start beyond video
end"))
202                         continue
203
204                     if end > video_duration:
205                         original_end = end
206                         end = video_duration
207                         logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
208                                     f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
209
210                     clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
211
212                 # Precise sub-second trim using fast re-encoding to preserve stream
headers
213                 padded_start = max(0.0, start - self.begin_padding)
214                 padded_end = end + self.end_padding
215
216                 # Clamp padded_end to video duration if known
217                 if video_duration > 0 and padded_end > video_duration:
218                     padded_end = video_duration
219                 logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
220                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
221
222                 trim_duration = padded_end - padded_start

```



```

223         logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
224
225         base_cmd = [
226             "ffmpeg", "-y",
227             "-ss", str(round(padded_start, 3)),
228             "-to", str(round(padded_end, 3)),
229             "-i", raw_video_path,
230             "-c:v", "libx264",
231             "-preset", "superfast",
232             "-crf", "22",
233             "-c:a", "aac",
234             "-b:a", "128k",
235         ]
236         vf_args = ["-vf", watermark_filter] if watermark_filter else []
237
238         # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
239         # path), retry once WITHOUT it so a branding misconfig can never nuke
the
240         # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
241         attempts = [vf_args, []] if vf_args else [[]]
242         produced = False
243         last_exc: Optional[subprocess.CallProcessError] = None
244         cmd = base_cmd + [clip_path]
245         for attempt_vf in attempts:
246             cmd = base_cmd + attempt_vf + [clip_path]
247             try:
248                 subprocess.run(cmd, capture_output=True, check=True)
249             except subprocess.CallProcessError as e:
250                 last_exc = e
251                 if attempt_vf and vf_args:
252                     logger.warning(f"{segment_label}: encode with watermark
failed; retrying without it.")
253                     continue
254                     break
255                 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
256                     produced = True
257                     if not attempt_vf and vf_args:
258                         logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
259                         break
260                     last_exc = None # ffmpeg succeeded but produced nothing
261                     break
262
263         if produced:
264             temp_clips.append(clip_path)
265         elif last_exc is not None:
266             stderr_msg = last_exc.stderr.decode(errors='replace').strip()
267             logger.error(f"{segment_label}: FFmpeg trim failed.")
268             print(f"  Command: {' '.join(cmd)}")
269             print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
270             skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
271             continue
272         else:
273             logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
274             skipped_segments.append((idx, start, end, "empty output clip"))
275             continue
276
277         # Report summary of skipped segments
278         if skipped_segments:

```

```

279         logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
280         for seg_idx, seg_start, seg_end, reason in skipped_segments:
281             logger.info(f"    - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
282             logger.info("")
283
284         if not temp_clips:
285             raise RuntimeError(
286                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
287                 f"All {len(segments)} segments were skipped. "
288                 f"Check the errors above for details. "
289                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
290             )
291
292         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
293
294         # Create a list file for FFmpeg concat demuxer
295         list_file_path = os.path.join(tmp_dir, "clips.txt")
296         with open(list_file_path, "w") as f:
297             for clip in temp_clips:
298                 # Escape single quotes in path
299                 safe_path = clip.replace("'", "'\\'")
300                 f.write(f"file '{safe_path}'\n")
301
302         logger.info("Stitching clips together...")
303         # Stitch them together without re-encoding (since they were encoded with
identical settings)
304         concat_cmd = [
305             "ffmpeg", "-y",
306             "-f", "concat",
307             "-safe", "0",
308             "-i", list_file_path,
309             "-c", "copy",
310             output_path
311         ]
312
313         try:
314             subprocess.run(concat_cmd, capture_output=True, check=True)
315
316             # Final verification
317             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
318                 raise RuntimeError(
319                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
320                     f"but output file is missing or empty: {output_path}"
321                 )
322
323             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
324             logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
325             return output_path
326         except subprocess.CalledProcessError as e:
327             stderr_msg = e.stderr.decode(errors='replace').strip()
328             raise RuntimeError(
329                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
330                 f"Command: {' '.join(concat_cmd)}\n"
331                 f"FFmpeg stderr: {stderr_msg[-500:]}"
332             ) from e
333

```

20. user (2026-06-12T09:08:42.406Z)

95 f1: float

```

96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104     def score(preds: List[Interval], gts: List[Interval],
105             iou_threshold: float = 0.5,
106             match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114         tp = len(mr.matches)
115         fp = len(mr.unmatched_pred_indices)
116         fn = len(mr.unmatched_gt_indices)
117
118         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
121 else 0.0
122         if mr.matches:
123             mean_iou = sum(m.iou for m in mr.matches) / tp
124             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
125 mr.matches) / tp
126             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
127 mr.matches) / tp
128         else:
129             mean_iou = mean_start_err = mean_end_err = 0.0
130
131         return Score(
132             iou_threshold=iou_threshold,
133             num_pred=len(preds), num_gt=len(gts),
134             true_positives=tp, false_positives=fp, false_negatives=fn,
135             precision=precision, recall=recall, f1=f1,
136             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
137         )
138
139     def pr_curve(preds: List[Interval], gts: List[Interval],
140             thresholds: Optional[List[float]] = None) -> List[Score]:
141         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
142
143         Lets you see strict-vs-lenient behaviour rather than a single operating point.
144         """
145         if thresholds is None:

```

21. user (2026-06-12T09:08:52.227Z)

```

1     # Roora Annotation Brief ? Rally Golden Set
2
3     **Project:** racket-sport rally segmentation golden / regression dataset
4     **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
5 doubles**)
6     **Companion doc:** [ANNOTATION_GUIDE.md](./ANNOTATION_GUIDE.md)
7
8     > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
9 that

```

```

8     > imports directly via `python -m src.cli.curate import-local`.
9
10    ## 0. What we need in one sentence
11    For each provided match video, produce one CSV file listing every rally with its
12    start time, end time, number of shots, and how it ended ? excluding all dead time.
13
14    ## 1. Deliverable format (canonical)
15    One CSV per video (UTF-8), named exactly `.csv` (we supply the `video_id`).
16    One row per rally, chronological, header row required.
17
18    | column | meaning |
19    |---|---|
20    | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21    | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22    | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23    | `start_time` | decimal seconds (e.g. `72.40`). Auto-filled from `start_mmss`.
24    | `end_time` | decimal seconds. Auto-filled from `end_mmss`. |
25    | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26    | `ending_reason` | one of: `winner` \| `forced_error` \| `unforced_error` \|
27    `service_fault` \| `let` \| `other` (why it ended) |
28    | `last_shot_outcome` | one of: `in` \| `net` \| `out` \| `n_a` (where the last shot
29    landed; `n_a` for service_fault/let) |
30    | `score_before` | optional, `"A-B"`. Blank if not confidently readable. |
31    | `score_after` | optional, `"A-B"`. |
32    | `notes` | optional free text. Required when `ending_reason = other`. |
33
34    Times (mm:ss ? decimal seconds). Raters type only `start_mmss`/`end_mmss`. The
35    template
36    Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
37    row 2):
38
39    ```
40    =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
41    VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
42    ```
43
44    On Download as CSV, the computed decimal seconds are exported. (If your tool already
45    shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
46    columns blank.)
47
48    Example (badminton):
49
50    ```csv
51    rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
52    _outcome,score_before,score_after,notes
53    1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
54    2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
55    3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
56    pressure)
57    ```
58
59    Critical format rules
60
61    - `start_time`/`end_time` that reach us must be decimal seconds (the formula
62    guarantees this).
63
64    - Times are relative to the start of the file we provide. Do not trim, clip, or
65    re-encode the video ? our pipeline identifies a video by its file checksum.
66
67    - `end_time` > `start_time` for every row.
68
69    - No overlaps: `end_time[N]` <= `start_time[N+1]`.
70
71    - Everything between one rally's `end_time` and the next rally's `start_time` is dead
72    time and gets no row.
73
74    ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
75
76    1. Ignore all dead time. Only label active play.
77
78    2. Include every rally ? exclude none. Every served point, including service
79    faults,

```

```

61      1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this
project.
62      **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
up looks
63      like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
serve,
64      cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls
65      in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
it.
66      3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
clips are not shots.
67
68      ## 3. Scope & phasing
69      **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
tennis),
70      each `< 3 min` with ~10?12 rallies. This is the **paid calibration round**; we review
jointly and
71      lock the Guide before scale-up.
72
73      **Phase B ? Scale (after a successful pilot):** ~12?15 videos **per sport** (~36?45
total), each
74      ~30 min. Same format, rules, and template.
75
76      > **Doubles are included** across the program. Rally rules are identical to singles; the
77      > badminton- and pickleball-specific doubles notes are in the Guide.
78
79      ## 4. Quality bar (acceptance criteria)
80      Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a
81      ~20% human spot-check. A file is returned for rework (in scope) if it misses:
82      - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
83      - **Boundaries:** `start_time`/`end_time` within **±0.3 s** of true serve-contact /
landing.
84      - **shots_count:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
85      - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
86      - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
87
88      ## 5. Per-sport quick reference (full version + examples in the Guide)
89      **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle
floors / fault · shot: each racket?shuttle contact.
90      **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double
bounce / out / net / fault · shot: each paddle?ball contact (floor bounces are **not** shots;
two-bounce rule is normal play).
91      **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal
return · shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
`let`.
92
93      ## 6. ending_reason values (decision tree in the Guide)
94      This vocabulary attributes **why** the point ended; it is shared across all
95      net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
96      the last shot physically landed in the separate **`last_shot_outcome`** column
97      (**`in` \ | `net` \ | `out` \ | `n_a`**) ? e.g. a forced error that flew long is
98      **`ending_reason=forced_error, last_shot_outcome=out`.
99
100     | value | meaning |
101     |---|---|
102     | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
winner). |
103     | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
stretched, wrong-footed, or rushed by the opponent's strong shot. |
104     | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
time and position to make. |
105     | `service_fault` | the serve itself failed / was illegal. |
106     | `let` | the point was replayed. |

```

```

107 | `other` | anything else / cannot tell. **Must** add a note. |
108
109 ## 7. Recommended capture method
110 Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
111 **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
112 (Label Studio / CVAT). The deliverable is the CSV in the schema above.
113 `seconds = frame_number / fps` (we provide fps per video).
114
115 ## 8. Logistics & contacts
116 - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117 - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[INTAKE_MANIFEST_TEMPLATE.csv](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118 - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
119 - Blank annotation template Sheet (mm:ss ? seconds formula from §1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120 - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121 - NDA before any footage is shared ? template [NDA_TEMPLATE.md](./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
122 - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124 ## 9. Pricing request (please quote)
125 1. Price **per rally** and **per minute** of footage.
126 2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127 3. Indicative **scale** price for ~36?45 longer videos.
128 4. Turnaround for the pilot and throughput (rallies/day) at scale.
129 5. QA process and rework policy.
130 6. Whether annotators have racket-sport familiarity (helpful, not required).
131 7. Confirmation you can deliver the CSV schema above.
132

```

22. user (2026-06-12T09:08:53.130Z)

```

1 # Delivery validation ? pilot_badminton_01
2
3 - rallies: **24**
4 - issues: **4** (blocker 1, high 0, medium 3, low 0)
5 - structurally-clean rallies: **21/24** (87%)
6 - gate: **BLOCKED**
7
8 ## BLOCKER
9 - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s ? rallies
must be disjoint.
10
11 ## MEDIUM
12 - (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low ? the span may
be inflated by a stray box.
13 - (shot_density) Rally 7: 2 shots over 16.0s (0.12/s) is implausibly low ? the span may
be inflated by a stray box.
14 - (shot_density) Rally 16: 2 shots over 17.5s (0.11/s) is implausibly low ? the span may
be inflated by a stray box.
15
16 ## Rallies to review: [2, 7, 16]

```

23. user (2026-06-12T09:08:53.334Z)

```

1 CVAT rally conversion report ? annotations.xml
2 fps used : 59.940
3 sampling step : 30 frames (~0.50s)
4 rallies parsed : 24

```

```

5         issues           : 5 (1 blocker)
6
7         [BLOCKER]
8         - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s ? rallies
must be disjoint.
9
10        [HIGH]
11        - (coarse_sampling) Frames sampled every 30 (~0.50s at 59.94fps); rally boundaries
carry +/-0.50s uncertainty, exceeding the +/-0.3s spec.
12
13        [MEDIUM]
14        - (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low ? the span
may be inflated by a stray box.
15        - (shot_density) Rally 7: 2 shots over 16.0s (0.12/s) is implausibly low ? the span
may be inflated by a stray box.
16        - (shot_density) Rally 16: 2 shots over 17.5s (0.11/s) is implausibly low ? the span
may be inflated by a stray box.
17

```

24. user (2026-06-12T09:09:09.608Z)

```

1         # Golden-Set Subproject ? Design & Implementation Overview
2
3         **Status:** In progress . **Date:** 2026-05-30
4         **Companion docs:** design rationale + vendor shortlist in
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in
[EVALUATION.md](EVALUATION.md).
5
6         This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8         > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10        ---
11
12        ## Architecture in one picture
13
14        ```
15                video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]  ground truth
16                ?           ?           ?           ?
17                FileProvider HumanVendor Automated(oracle) ?
18                (GS-1)       (GS-3)       (GS-4)
backend.eval.metrics.score() ??? existing, unchanged
19
20                production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]  predictions
21
22                                regress(video, tier) (GS-2)
compares the two
23        ```
24
25        **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]`** that
`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26

```

```

27      ---
28
29      ## PR breakdown
30
31      | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32      |---|---|---|---|---|
33      | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34      | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35      | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36      | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37      | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39      Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41      ---
42
43      ## Slice details
44
45      ### GS-1 ? Provider registry + FileProvider ? (this PR)
46      - `backend/annotations/base.py` ? `AnnotationProvider` ABC (`submit`/`poll`/`fetch` +
`annotate()` convenience) and `annotation_registry`, mirroring `backend/providers` and
`backend/indexer` registry patterns.
47      - `backend/annotations/file_provider.py` ? `FileProvider`: the "job" is a CSV that
already exists (formalizes today's manual/ShuttleSet flow); reuses
`eval.annotations.load_annotations`.
48      - Tests assert the registry contract **and** the keystone (provider output is directly
scorable by `metrics.score()`).
49      - **Design choice:** `fetch()`/`annotate()` raise loudly on a missing/incomplete job
rather than returning `[]`, so a broken label source can't silently masquerade as "0 rallies"
and skew a regression to green.
50
51      ### GS-2 ? Regression runner (next)
52      `regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when
precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
53
54      ### GS-3 ? HumanVendorProvider (after your vendor pick)
55      A concrete provider that submits a (licensed/owned-only) video to the chosen vendor and
fetches the returned rally CSV. Carries the privacy controls from the design doc (secure time-
limited links, delete-on-delivery, DPA). Thin adapter ? the interface already exists from GS-1.
56
57      ### GS-4 ? AutomatedProvider (fast-tier oracle) ?? oracle rule
58      An independent strong labeler for the fast/CI tier. **Must be a different model lineage
than whatever production uses** (else shared blind spots ? false green). A fast-tier failure is
a *smoke alarm* that escalates to the GS-3 human tier, never an authoritative verdict on its
own.
59
60      ### GS-5 ? Regression triggers
61      Wire `regress()` into CI (pre-merge gate on pipeline changes), an on-demand CLI (the
"surprise regression"), and an optional scheduled run. Includes a deliberate re-snapshot path so
legitimate production improvements can rebaseline instead of raising false regressions.
62
63      ---
64
65      ## Conventions

```


66 - ****Branch per slice****, one PR each, tests green before PR, full suite must stay green.
67 - New runtime deps: ****none**** through GS-2 (GS-3 may add a vendor SDK; GS-4 may reuse the
existing `google-genai` for a Gemini oracle).
68 - All ground-truth video stays ****licensed/owned**** ? see the privacy section of the
design doc.
69

25. user (2026-06-12T09:09:11.160Z)

```
backend\eval\calibrate_local.py:47:         points, fps=fps, frame_width=frame_width,  
backend\eval\calibrate_wasb.py:51:         points, fps=fps, frame_width=frame_width,  
backend\eval\distill_local.py:53:         points, fps=fps, frame_width=frame_width,  
backend\eval\distill_local.py:75:         points, fps=fps, frame_width=frame_width,  
backend\pipeline\segmenters\trajectory_hybrid.py:162:         points, fps=fps,  
frame_width=frame_width, chunk_start=0.0,
```

26. user (2026-06-12T09:09:12.587Z)

No matches found

27. user (2026-06-12T09:09:27.168Z)

```
75  
76     ### Rule 3: A shot = one contact with the racket / paddle / bat  
77     Count every time the shuttle/ball touches a racket, paddle, or bat. The serve is shot 1.  
78     - **Count:** any racket/paddle/bat contact (serve, return, smash, block?).  
79     - **Don't count:** floor bounces, table bounces, net clips, the toss before a serve.  
80  
81     5-shot example: `serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5)` ? `shots_count =  
5`, `ending_reason = winner`.  
82  
83     ## Part 3 ? Where exactly does a rally start and end?  
84     **START (`start_mmss`):** the exact frame the server's racket/paddle/bat **contacts**  
the  
85     shuttle/ball. Not the toss, not the wind-up ? the contact frame.  
86     `[SCREENSHOT: serve-contact frame]`  
87  
88     **END (`end_mmss`):** the exact frame the shuttle/ball becomes **dead**:  
89     - Badminton: shuttle touches the floor, or play stops on a fault.  
90     - Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.  
91     - Table tennis: the second bounce on one side, a non-serve net hit, or off the table.  
92  
93     `[SCREENSHOT: end frame]`  
94  
95     If occluded/off-screen, pick the closest clear frame and add a note.  
96     **Boundary tolerance:  $\pm 0.3$  s** (~ $\pm 9$  frames at 30 fps).  
97  
98     ## Part 4 ? ending_reason: how to choose  
99     Pick exactly **one** value for how the rally ended. Stop at the first match:  
100  
101     1. Did the **serve** itself fail / was illegal (and ended the point)? ? `service_fault`  
102     2. Was the point **replayed** (let / interruption)? ? `let`  
103     3. Did the final shot land **in** and the opponent make **no legal contact**? ? `winner`  
(a clean winner)  
104     4. Did the losing player make an error (netted / out / missed) **under pressure** ?  
105     stretched, wrong-footed, or rushed by a strong shot? ? `forced_error`  
106     5. Did the losing player make an error on a **routine, unpressured** ball they had  
107     the time and position to make? ? `unforced_error`  
108     6. None of the above / cannot tell ? `other` (add a note)  
109  
110     **Forced vs unforced ? the one judgment call.** Decide using the opponent's **preceding**  
111     shot:  
112     - `forced_error` = the opponent's shot **caused** the miss (pace, placement, deception;  
113     the player barely reached it or was pulled out of position).
```

```
114 - `unforced_error` = a clean, makeable ball the player missed on their own.
115 - When genuinely unsure between the two, choose `unforced_error` and add a brief note.
116
117 Notes:
118 - A clean `winner` means **no legal contact** by the opponent. If they got a racket /
paddle /
119 bat on it but netted or hit out, that is a `forced_error` / `unforced_error` (their
shot), not a `winner`.
120 - A serve into the net is `service_fault` (rule 1 beats everything).
121
122 ### Then set `last_shot_outcome` ? where the last shot physically went
123 This is a **separate, observational** column (no judgment): just look at the last shot.
124 - `in` ? landed in (winners; also a put-away the opponent couldn't reach).
```

28. user (2026-06-12T09:09:28.110Z)

```
30 table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31 `is_commercial`, so only licensable data flows
32 downstream. The annotations are **temporal-only** ? they train/evaluate rally
33 *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35 - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36 the DB **in place** so annotations and curation status are never lost. Uses
37 authenticated cookies for reliable, unthrottled downloads. The why and the
38 hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40 ## The one contract that matters: video identity
41
42 The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43 re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44 exact `local_path`, and any downstream processing must run on **that** file. This is
45 why acquisition (`fetch`) happens *before* the indexer processes anything.
46
```

29. user (2026-06-12T09:09:28.492Z)

```
1 video_id,sport,format,match_name,source_filename,drive_link,fps,duration_mmss,expected_r
allies,assigned_to,status,notes
2 pilot_badminton_01,badminton,singles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:45,10-12,,todo,deliver as pilot_badminton_01.csv
3 pilot_pickleball_01,pickleball,doubles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:30,10-12,,todo,deliver as pilot_pickleball_01.csv
4 pilot_tabletennis_01,tabletennis,singles,TODO(avidullu): match name,TODO(avidullu):
exact source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:20,10-12,,todo,deliver as pilot_tabletennis_01.csv
5
```

30. user (2026-06-12T09:09:40.043Z)

```
10:[Omitted long matching line]
48:[Omitted long matching line]
152:| **Roora** (Roora ML Pvt Ltd) | **Bengaluru** ? HSR Layout (verified, `rooragrp.com`) |
Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot
(2026-06-02)** ? existence/capability/location verified; **quality TBD via the pilot**. |
**Verified** (capability) · pilot pending |
164:> **Update (2026-06-02):** the founder opted to start with **Roora** (Bangalore-local,
startup-friendly) as a **single-vendor pilot** rather than the iMerit/Taskmonk multi-vendor
bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot
doesn't clear the $3/$4 quality bar.
182:[Omitted long matching line]
```

31. user (2026-06-12T09:09:41.294Z)

```

badminton
manifest.json
{
  "commercial_only": true,
  "statuses": [
    "curated"
  ],
  "total_videos": 7,
  "total_segments": 264,
  "eval_sets": [
    {
      "video_id": "my_custom_local_match",
      "sport": "badminton",
      "csv": "badminton/my_custom_local_match.csv",
      "local_path": "data/videos/my_custom_match.mp4",
      "video_url": null,
      "match_name": "My Custom Local Match",
      "fps": 30.0,
      "resolution": "1080p",
      "license_type": "Proprietary",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 2
    },
    {
      "video_id": "adarsh_avi_singles",
      "sport": "badminton",
      "csv": "badminton/adarsh_avi_singles.csv",
      "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\Adarsh and Avi.MP4",
      "video_url": null,
      "match_name": "Adarsh and Avi (singles)",
      "fps": 59.94,
      "resolution": "1080p",
      "license_type": "Proprietary",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 96
    },
    {
      "video_id": "kushagra_singles",
      "sport": "badminton",
      "csv": "badminton/kushagra_singles.csv",
      "local_path": "C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\KushagraYashAviFirstVideo.MP4",
      "video_url": null,
      "match_name": "Kushagra Yash Avi (singles, multi-court)",
      "fps": 59.94,
      "resolution": "4K",
      "license_type": "Proprietary",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 32
    },
  ],
}

```

32. user (2026-06-12T09:09:48.698Z)

```

backend\\eval\\distill_local.py:21:   python -m backend.eval.distill_local --manifest videos.json
--model-out output/local_rally_model.json
backend\\eval\\distill_local.py:148:   ap.add_argument("--manifest", required=True,
backend\\eval\\distill_local.py:154:   with open(args.manifest, encoding="utf-8") as f:
backend\\eval\\calibrate_local.py:19:   python -m backend.eval.calibrate_local --manifest
videos.json
backend\\eval\\calibrate_local.py:115:   ap.add_argument("--manifest", required=True,
backend\\eval\\calibrate_local.py:120:   with open(args.manifest, encoding="utf-8") as f:

```

```

backend\eval\batch.py:1: """Batch evaluation across a collector export manifest.
backend\eval\batch.py:3: Phase 3 of the collector->indexer bridge: walk the `manifest.json`
emitted by
backend\eval\batch.py:22: def resolve_video_path(local_path: Optional[str], videos_root: str) ->
Optional[str]:
backend\eval\batch.py:23:     """Resolve a manifest `local_path` to an absolute path.
backend\eval\batch.py:30:     if not local_path:
backend\eval\batch.py:32:     p = local_path.replace("\\", "/")
backend\eval\batch.py:37:     # This keeps collector manifest paths working in tests and mixed
environments.
backend\eval\rally_seq_proto.py:26:     python -m backend.eval.rally_seq_proto --manifest
output/human_lovo_manifest.json
backend\eval\rally_seq_proto.py:154:     ap.add_argument("--manifest", required=True, help="JSON:
[{name, trajectory, fps, frame_width, labels}, ...]")
backend\eval\rally_seq_proto.py:156:     with open(args.manifest, encoding="utf-8") as f:

```

33. user (2026-06-12T09:09:58.565Z)

```

1     """Batch evaluation across a collector export manifest.
2
3     Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4     sports-data-collector's `curate export`, (optionally) process each video through
5     a segmenter so detections land in the indexer DB, then score every video's
6     detected rallies against its ground-truth CSV and aggregate the results.
7
8     This module holds the *pure* pieces (path resolution, stage selection, metric
9     aggregation) so they can be unit-tested without touching video files, the GPU,
10    or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11    """
12
13    import os
14    from typing import Dict, List, Optional
15
16
17    # Segmenters that only ever populate the `final` play_segments table (no raw
18    # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19    _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22    def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23        """Resolve a manifest `local_path` to an absolute path.
24
25        The collector stores paths like `./data/videos/shuttle1.mp4` relative
26        to its own repo root and with Windows separators. Normalise separators, strip
27        a leading `./`, and resolve relative paths against `videos_root` (the
28        collector root). Returns None for a missing/empty path.
29        """
30        if not local_path:
31            return None
32        p = local_path.replace("\\", "/")
33        if p.startswith("./"):
34            p = p[2:]
35        # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36        # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37        # This keeps collector manifest paths working in tests and mixed environments.
38        if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39            return os.path.normpath(p)
40        return os.path.normpath(os.path.join(videos_root, p))
41
42
43    def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44        """Pick which prediction stages to score.
45
46        `auto` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47        but only `final` for segmenters that don't emit candidates. An explicit

```

```

48     request ('candidates' | 'final' | 'both') overrides the heuristic.
49     """
50     if requested == "both":
51         return ["candidates", "final"]
52     if requested in ("candidates", "final"):
53         return [requested]
54     # auto
55     if segmenter in _FINAL_ONLY_SEGMENTERS:
56         return ["final"]
57     return ["candidates", "final"]
58
59
60 def _safe_div(num: float, den: float) -> float:
61     return num / den if den else 0.0
62
63
64 def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65     """Aggregate per-video stage scores into corpus-level metrics.
66
67     `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
68     ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
69     compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70     a TP-weighted mean IoU, plus the macro mean of per-video F1.
71     """
72     out: Dict[str, dict] = {}
73     for stage in stages:
74         tp = fp = fn = 0
75         iou_weighted = 0.0
76         f1s: List[float] = []
77         n_videos = 0
78         for rep in per_video:
79             sr = rep.get("stages", {}).get(stage)
80             if not sr:
81                 continue
82             s = sr["score"]
83             tp += s["true_positives"]
84             fp += s["false_positives"]
85             fn += s["false_negatives"]
86             iou_weighted += s["mean_iou"] * s["true_positives"]
87             f1s.append(s["f1"])
88             n_videos += 1
89         precision = _safe_div(tp, tp + fp)
90         recall = _safe_div(tp, tp + fn)

```

34. user (2026-06-12T09:10:08.993Z)

```

backend\main.py:508:     compiler = VideoCompiler(output_dir, watermark=watermark)
backend\main.py:633:     compiler = VideoCompiler(out_dir,
watermark=load_app_config().stitching.watermark)
backend\config\models.py:142:     begin_padding: float = 0.5
backend\config\models.py:143:     end_padding: float = 0.5
backend\pipeline\stitcher\compiler.py:10:     def __init__(self, output_dir: str, end_padding:
float = 1.0, begin_padding: float = 0.5,
backend\pipeline\stitcher\compiler.py:13:         self.end_padding = end_padding
backend\pipeline\stitcher\compiler.py:14:         self.begin_padding = begin_padding
backend\pipeline\stitcher\compiler.py:148:         into a seamless highlights file, applying
end_padding to prevent abrupt endings.
backend\pipeline\stitcher\compiler.py:173:         logger.info(f"Trimming {len(segments)}
segments (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
backend\pipeline\stitcher\compiler.py:213:         padded_start = max(0.0, start -
self.begin_padding)
backend\pipeline\stitcher\compiler.py:214:         padded_end = end + self.end_padding
backend\pipeline\stitcher\compiler.py:219:         logger.info(f"{segment_label}:
Padded end ({end + self.end_padding:.2f}s) exceeds video duration. "
backend\pipeline\segmenters\gemini.py:407:         begin_pad = stitch_cfg.get("begin_padding",

```

```
0.5)
backend\pipeline\segmenters\gemini.py:408:         end_pad = stitch_cfg.get("end_padding", 0.5)
```

35. user (2026-06-12T09:10:09.709Z)

```
100     PATH yet. We add it here so `fetch` works without requiring a terminal
101     restart ? without clobbering a deno the user already has elsewhere.
102     """
103     env = os.environ.copy()
104     if shutil.which("deno"):
105         return env # already discoverable
106     deno_bin = os.path.join(os.path.expanduser("~"), ".deno", "bin")
107     if os.path.isdir(deno_bin):
108         env["PATH"] = deno_bin + os.pathsep + env.get("PATH", "")
109     return env
110
111
112     def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int],
Optional[float]]:
113         """Return (width, height, fps) of a video file via ffprobe, best-effort.
114
115         Returns (None, None, None) if ffprobe is unavailable or the file can't be
116         parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
117         """
118         if not shutil.which("ffprobe") or not os.path.exists(path):
119             return (None, None, None)
120         try:
121             out = subprocess.run(
122                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
123                  "-show_entries", "stream=width,height,avg_frame_rate",
124                  "-of", "default=noprint_wrappers=1:nokey=1", path],
125                 stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
126             ).stdout.split()
127         except (subprocess.SubprocessError, OSError):
128             return (None, None, None)
129
130         width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
131         height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
132         fps = None
133         if len(out) > 2 and "/" in out[2]:
134             num, den = out[2].split("/")
135             try:
136                 den_f = float(den)
137                 fps = round(float(num) / den_f, 3) if den_f else None
138             except ValueError:
139                 fps = None
140         return (width, height, fps)
141
142
143     @dataclass
144     class DownloadResult:
145         success: bool
146         path: Optional[str] = None
147         size_mb: float = 0.0
148         width: Optional[int] = None
149         height: Optional[int] = None
```

36. user (2026-06-12T09:10:21.092Z)

```
495     # Reject path traversal / absolute output names (os.path.join would otherwise let an
496     # absolute or '..'-laden output_filename write anywhere on disk).
497     try:
498         out_name = safe_output_filename(req.output_filename)
499     except ValueError as e:
500         raise HTTPException(status_code=400, detail=str(e)) from e
```

```

501
502     # Extract start/end time pairs
503     time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
504
505     # Run compiler (with the optional configured branding watermark, if enabled)
506     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
507     or "output"
508     watermark = getattr(getattr(global_config, "stitching", None), "watermark", None)
509     compiler = VideoCompiler(output_dir, watermark=watermark)
510     try:
511         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
512         return {"status": "success", "filepath": out_path}
513     except Exception as e:
514         raise HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}") from e
515
516     # --- CLI Debug Utility & Orchestration ---
517     def run_cli_diagnose_clip(video_path: str, timestamp: float):
518         """CLI debugging utility to examine segment properties around a timestamp."""
519         print("=" * 60)
520         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")

```

37. user (2026-06-12T09:10:22.098Z)

```

395         non_overlapping[-1] = curr # Replace previous with current
396     else:
397         non_overlapping.append(curr)
398     validated = non_overlapping
399
400     # Log validation summary
401     if rejected:
402         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
403         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
404 "\n".join(reject_lines))
405     logger.info(f"{len(validated)} play segments passed validation (out of {len(ai_segments)} from Gemini).")
406
407     stitch_cfg = self.config.get("stitching", {})
408     begin_pad = stitch_cfg.get("begin_padding", 0.5)
409     end_pad = stitch_cfg.get("end_padding", 0.5)
410
411     total_highlight_dur = 0.0
412     for c in validated:
413         total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
414
415     mins = int(total_highlight_dur // 60)
416     secs = int(total_highlight_dur % 60)
417     logger.info(
418         f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
419         f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin + {end_pad}s end padding per segment)"
420     )
421
422     # 9. Populate SQLite DB with validated play segments
423     for c in validated:
424         start = c["start"]
425         end = c["end"]

```

38. user (2026-06-12T09:10:32.627Z)

```

1     """Normalize a VLC rally-annotator CSV into a clean lean CSV that ``import-local``
2     ingests safely.
3
4     Two verified landmines in PARTIAL human annotation data (a video tagged only up to
5     some point, with mid-session corrections):

```

```

6
7     1. An unusable time aborts the WHOLE import (one bad row kills the entire video,
8       via the try/except around _parse_local_annotations in curate.py import-local):
9       a blank/missing time makes ``safe_float_required`` raise; a reversed
10      ``end <= start`` or a negative ``start`` makes the ``BadmintonRally`` pydantic
11      validator raise. -> we DROP such rows, mirroring the downstream indexer loader
12      which already skips ``not end > start``.
13     2. A fractional or duplicate ``rally_number`` (e.g. ``7.5`` inserted for a rally
14       the annotator missed) truncates via ``int(float())`` and COLLIDES on the
15       ``(video_id, rally_number)`` primary key; the DELETE-then-INSERT upsert then
16       silently destroys one rally. -> we RENUMBER 1..N by start_time, recording the
17       original id for traceability.
18
19     It also accepts mm:ss / HH:MM:SS times (converted to decimal seconds) and folds a
20     pipe-delimited ``ending_reason`` ('forced_error|other') to its primary token, so the
21     output is always the canonical lean schema::
22
23         rally_number,start_time,end_time,ending_reason,sport[,shots_count]
24
25     plus traceability columns (``original_rally_number``, ``notes``) that import-local
26     ignores. Standalone on purpose: it does NOT touch the DB schema or the
27     rally_number-ordered export. Run it, then feed the output to ``curate import-local``.
28
29     python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
30
31     from __future__ import annotations
32
33     import argparse
34     import csv
35     import os
36     from dataclasses import dataclass, field
37     from typing import Any, Dict, List, Optional, Tuple
38
39     # Canonical lean output columns + traceability (import-local reads the first four/
40     # five; original_rally_number + notes are ignored by the importer but kept for humans).
41     OUT_FIELDS = ["rally_number", "start_time", "end_time", "ending_reason", "sport",
42                  "shots_count", "original_rally_number", "notes"]
43
44
45     @dataclass
46     class NormalizeReport:
47         rows_in: int = 0
48         rows_out: int = 0
49         dropped: List[Dict[str, Any]] = field(default_factory=list) # {orig_id, reason,
start, end}
50         renumbered: bool = False
51         fractional_ids: List[str] = field(default_factory=list)
52         duplicate_ids: List[str] = field(default_factory=list)
53         mmss_converted: int = 0
54
55     def summary(self) -> str:
56         lines = [f"rows in: {self.rows_in} -> rows out: {self.rows_out}"]
57         if self.dropped:
58             lines.append(f"dropped {len(self.dropped)} bad row(s):")
59             for d in self.dropped:
60                 lines.append(f"  - orig id {d['orig_id']!r}: {d['reason']} "
61                             f"(start={d['start']!r}, end={d['end']!r})")
62         if self.renumbered:
63             extra = []
64             if self.fractional_ids:
65                 extra.append(f"fractional ids {self.fractional_ids}")
66             if self.duplicate_ids:
67                 extra.append(f"duplicate ids {self.duplicate_ids}")
68             lines.append("renumbered rallies 1..N by start_time"
69                         + (f" ({'; '.join(extra)})" if extra else ""))

```



```

70         if self.mmss_converted:
71             lines.append(f"converted {self.mmss_converted} mm:ss timestamp(s) to
seconds")
72         if not self.dropped and not self.renumbered and not self.mmss_converted:
73             lines.append("clean input ? no changes beyond column normalization")
74         return "\n".join(lines)
75
76
77     def _to_seconds(raw: Any) -> Optional[Tuple[float, bool]]:
78         """(seconds, was_mmss) or None if blank/unparseable. Decimal seconds or mm:ss /
HH:MM:SS."""
79         if raw is None:
80             return None

```

39. user (2026-06-12T09:10:33.694Z)

```

1     from backend.pipeline.validators.base import VideoValidator, ValidationResult
2     from typing import Dict, Any
3
4     class ResolutionFPSValidator(VideoValidator):
5         @property
6         def name(self) -> str:
7             return "resolution_fps_check"
8
9         @property
10        def description(self) -> str:
11            return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14            meta = context.get("ffprobe_meta")
15            if not meta or not meta.get("streams"):
16                return ValidationResult(
17                    validator_name=self.name,
18                    passed=False,
19                    message="Video metadata is missing. Run format_check validator first."
20                )
21
22            stream = meta["streams"][0]
23            width = int(stream.get("width", 0))
24            height = int(stream.get("height", 0))
25
26            # Parse frame rate (e.g. "30/1" or "30000/1001")
27            fps_str = stream.get("r_frame_rate", "0/1")
28            try:
29                num, den = map(int, fps_str.split("/"))
30                fps = num / den if den != 0 else 0
31            except ValueError:
32                fps = 0.0
33
34            details = {
35                "width": width,
36                "height": height,
37                "fps": round(fps, 2)
38            }
39
40            # Check bounds from config

```

40. user (2026-06-12T09:11:09.403Z)

docs\CODE_MAP.md:155:- `@backend/utils/video.py` ? `::get\_video\_duration` (ffprobe; ? 0.0 on failure = "unknown"), `::extract\_video\_chunk(...,reencode=False,scale\_width=None)` . ? copy mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale\_width` requires `reencode=True` .

```

backend\utils\video.py:20:def extract_video_chunk(video_path: str, start_time: float, duration:
float,
backend\pipeline\segmenters\gemini.py:8:from backend.utils.video import get_video_duration,
extract_video_chunk
backend\pipeline\segmenters\gemini.py:146:                success = extract_video_chunk(
backend\pipeline\segmenters\trajectory_hybrid.py:29:from backend.utils.video import
get_video_duration, extract_video_chunk
backend\pipeline\segmenters\trajectory_hybrid.py:206:                if not
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
backend\pipeline\segmenters\yolo_hybrid.py:22:from backend.utils.video import
get_video_duration, extract_video_chunk
backend\pipeline\segmenters\yolo_hybrid.py:356:                success =
extract_video_chunk(video_path, cs, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:396:                success =
extract_video_chunk(video_path, chunk_start, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py:491:                success =
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True)
backend\eval\gemini_refine.py:11:sports.badminton prompt (sport_registry),
utils.video.extract_video_chunk,
backend\eval\gemini_refine.py:32:from backend.utils.video import get_video_duration,
extract_video_chunk
backend\eval\gemini_refine.py:83:    if not extract_video_chunk(video, cs, ce - cs, clip,
reencode=True, scale_width=clip_scale_width):
backend\eval\gemini_refine.py:171:        if not extract_video_chunk(video, cstart, cend -
cstart, clip, reencode=True, scale_width=clip_scale_width):
tests\test_gemini.py:41:@patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
tests\test_gemini.py:102:@patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
tests\test_gemini.py:120:@patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
tests\test_gemini.py:139:@patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
tests\test_tracknet_hybrid.py:30:    fn =
patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk")(fn)
tests\test_trajectory_hybrid_equivalence.py:51:
patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk", return_value=True), \
tests\test_video.py:2:from backend.utils.video import get_video_duration, extract_video_chunk
tests\test_video.py:17:def test_extract_video_chunk_success(mock_run):
tests\test_video.py:18:    success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
tests\test_video.py:23:def test_extract_video_chunk_failure(mock_run):
tests\test_video.py:25:    success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
tests\test_yolo_hybrid.py:45:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk
")
tests\test_yolo_hybrid.py:91:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk
")
tests\test_yolo_hybrid.py:111:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chun
k")
tests\test_yolo_hybrid.py:181:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chun
k")
tests\test_yolo_hybrid.py:232:@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chun
k")

```

41. user (2026-06-12T09:11:10.386Z)

```

142:         "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143:         "-c:a", "aac", "-b:a", "128k",

```

42. user (2026-06-12T09:11:11.587Z)

```

58:         max_gap = 0.0
59:         backward_jumps = 0
66:             backward_jumps += 1
67:             if gap > max_gap:
68:                 max_gap = gap
71:         # If there's a keyframe gap larger than 8.0 seconds or a negative gap (backward
jump), it's highly likely spliced.
74:         "max_keyframe_gap_seconds": round(max_gap, 2),
75:         "backward_jumps_detected": backward_jumps,

```

```

80:         if backward_jumps > 0:
88:         # Threshold for sudden gap (e.g. 10.0 seconds)
89:         if max_gap > 10.0:
93:             message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
100:             message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",

```

43. user (2026-06-12T09:11:20.546Z)

```

135
136         os.makedirs(chunks_dir, exist_ok=True)
137
138         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
139             chunk_end = min(chunk_start + chunk_duration, video_duration)
140             actual_chunk_dur = chunk_end - chunk_start
141             chunk_label = f"Chunk {ci+1}/{num_chunks}"
142             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
143
144             # Extract chunk with ffmpeg
145             logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
146             success = extract_video_chunk(
147                 video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
148                 reencode=ai_scale_width > 0,
149                 scale_width=ai_scale_width if ai_scale_width > 0 else None,
150             )
151             if not success:
152                 logger.error(f"Failed to extract {chunk_label}. Skipping.")
153                 return chunk_label, [], {}
154
155             # Measure chunk size
156             try:
157                 chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
158             except Exception:
159                 chunk_size_mb = 0.0

```

44. user (2026-06-12T09:11:21.211Z)

```

60         st = float(sg["start_time"]) + clip_start
61         en = float(sg["end_time"]) + clip_start
62     except (TypeError, ValueError, KeyError):
63         continue
64     st = max(clip_start, st)
65     en = min(clip_end, en)
66     if en > st:
67         out.append((round(st, 3), round(en, 3)))
68     return out
69
70
71     def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
72                       pad: float, duration: float, tmpdir: str, idx: int,
73                       clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
List[Interval]]:
74         # Own provider/client per call: the genai client isn't reliably thread-safe, and a
75         # shared one across workers throws socket errors. A fresh lazy client is cheap.
76         provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
77         s, e = window
78         cs = max(0.0, s - pad)
79         ce = min(duration, e + pad) if duration > 0 else e + pad
80         clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
81         # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
82         # 500MB upload cap even for 4K/5K sources.
83         if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,

```

```

scale_width=clip_scale_width):
84         return idx, window, []
85     try:
86         prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
87         raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
88         mapped = _offset_segments(raw, cs, ce)
89     finally:
90         try:
91             os.remove(clip)
92         except OSError:
93             pass
94     # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
95     # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
swept
96     # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
later
97     # by the gap-tolerant global merge in run().
98     return idx, window, mapped
99

```

45. user (2026-06-12T09:11:32.689Z)

```

backend\pipeline\segmenters\gemini.py:117:         ai_scale_width =
int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
backend\pipeline\segmenters\gemini.py:148:         reencode=ai_scale_width > 0,
backend\pipeline\segmenters\gemini.py:149:         scale_width=ai_scale_width if
ai_scale_width > 0 else None,

```

46. user (2026-06-12T09:11:33.711Z)

```

1     # Rally Ingestion Pipeline ? End-to-End FAQ
2
3     How a vendor's rally annotations become training/eval ground truth, and how we
4     validate each delivery against our conventions. This is the operational companion
5     to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7     ```
8     vendor delivery                OUR side (collector)                modeling
(indexer)
9     ?????????????????? convert  ?????????????????? validate ??????????????????
10    ? CVAT export ? ?????????? ? canonical CSV ? ?????????? ? report .md ?
11    ? (boxes + attrs)?           ? (per rally, ?           ? report .json ?
12    ? OR ?                       ? decimal secs)?         ? review .csv ??? human
verifies
13    ? canonical CSV ?           ??????????????????         ??????????????????
flagged rallies
14    ??????????????????         ?                             ?
15                                ? import-local (validated) ?
16                                ?                             golden labels
17    ?????????????????? export  ??????????????????
18    ? SQLite (rally ? ?????????? ? <id>.csv ?
start,end (indexer GT)
19                                ? tables) ?                 ? manifest.json?
20    ??????????????????         ??????????????????
21                                ?
22                                slice clips / calibrate /
train
23    ```
24
25    ## The four commands (in order)
26
27    ```bash
28    # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)

```

```

29     python -m src.cli.curate convert-cvat --input job.xml \
30         --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32     # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33     python -m src.cli.curate validate-delivery --input job.xml \
34         --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35     # ...or validate a CSV directly:
36     python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38     # 3. After human review, import the (corrected) CSV into the DB
39     python -m src.cli.curate import-local --sport badminton \
40         --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42     # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43     python -m src.cli.curate export --out-dir data/eval_export
44     ```
45
46     ---
47
48     ## FAQ
49
50     ### What delivery formats do we accept?
51     Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52     attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
53     as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54     CSVs.
55
56     ### What is the "canonical" CSV?
57     One row per rally:
58     `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
59     last_shot_outcome, score_before, score_after, notes`.
60     `start_time`/`end_time` are decimal seconds; `import-local` reads those.
61
62     ### How are timestamps handled? (the fps thing)
63     We recompute each rally's interval from its frame range × the true fps
64     (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
65     step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
66     from `--fps` or ffmpeg of `--video-path`. `mm:ss` is also accepted everywhere.
67
68     ### What does validation check?
69     | code | severity | meaning |
70     |---|---|---|
71     | `overlap` | blocker | two rallies' intervals overlap (`end[N] > start[N+1]`) |
72     | `zero_duration` | blocker | `end <= start` (single-sample / stray box) |
73     | `frame_out_of_range` | high | a box frame is past the video's last frame |
74     | `shot_density` | medium | long rally with implausibly few shots ? likely an inflated
span** (often hides a merged rally) |
75     | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an omitted rally |
76     | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
77     | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
78
79     ### Blocker vs. review ? what's the gate?
80     `validate-delivery` exits non-zero if any blocker exists (so it can gate a bulk
loop / CI). Non-blocker issues don't stop the gate but populate the review sheet

```

47. user (2026-06-12T09:12:00.669Z)

```

1: """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
6: strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
29:     """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
34:     cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35:         "-c:a", "pcm_s16le", out_wav]

```

48. user (2026-06-12T09:12:01.948Z)

```
63:     def _guard_import(self, sport_str, video_id, annotations, force=False):
83:         # annotations (filename-derived video_ids collide easily).
84:         if self.db.get_video(video_id):
86:             n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
90:             print(f"Error: video_id '{video_id}' already has {n} annotated segment(s); ")
95:             print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
114:         video_id = args.video_id or match_name.lower().replace(" ", "_")
139:         annotations = self._parse_local_annotations(sport_str, video_id,
annotations_path)
166:         self._guard_import(sport_str, video_id, annotations, force=getattr(args, "force",
False))
170:         video_id=video_id,
```

[Showing results with pagination = limit: 10]

49. user (2026-06-12T09:14:01.785Z)

Structured output provided successfully

50. assistant (2026-06-12T09:14:06.658Z)

Verification complete. Structured output returned.